

Tài liệu Thiết kế Kỹ thuật & Phần mềm (TDD/SDD) - Slopee Web3 Escrow MVP

Mã tài liệu: SDD-SLOPEE-MVP-01 · **Phiên bản:** v1.2 (thanh toán gom nhiều shop, chuyển VND, chốt Q-06 đến Q-10) · **Ngày:** 10/10/2026 **Căn cứ:** BRD-SLOPEE-MVP-01 v5.1 và SRS-SLOPEE-MVP-01 v5.1

1. Giới thiệu

1.1. Mục đích

BRD trả lời câu hỏi *vì sao và cái gì* về nghiệp vụ; SRS trả lời *hệ thống phải làm gì*. Tài liệu này trả lời *xây như thế nào*: phân rã thành phần, giao diện nội bộ, thuật toán, luồng xử lý, quyết định kỹ thuật và thiết kế kiểm thử, đủ để từng thành viên nhóm bắt tay lập trình mà không phải tự suy đoán.

Mã Solidity tham chiếu ở SRS mục 10 và lược đồ MySQL ở SRS mục 9 vẫn là nguồn chuẩn; tài liệu này không chép lại mà thiết kế xung quanh chúng, đồng thời nêu các điểm cần chỉnh khi rà soát (mục 15).

1.2. Phạm vi và đối tượng đọc

Đối tượng	Dùng tài liệu để
Thành viên A (Contract, bảo mật)	Mục 4, 8, 11, 13
Thành viên B (Backend, Indexer)	Mục 3, 5, 6, 7, 8, 10
Thành viên C (Frontend, kiểm thử E2E, tài liệu)	Mục 9, 10, 12, 13
Hội đồng / người rà soát	Mục 2, 3, 11, 14, 15

Phạm vi: mô-đun Web3 của Slopee (smart contract, indexer, API Flask bổ sung, giao diện React) theo đúng ranh giới MVP của BRD mục 2. Fiat, COD, SPayLater, CA thật, mainnet nằm ngoài phạm vi.

1.3. Quy ước

- **DD-xx**: quyết định thiết kế; **MOD-xx**: mô-đun; **SEQ-xx**: luồng tuần tự; **INV-xx**: bất biến; **R-xx**: phát hiện khi rà soát.
- Tiền luôn là số nguyên đơn vị nhỏ nhất của token (uint256 / chuỗi thập phân), tuyệt đối không dùng số thực.
- Thời gian lưu UTC; thời gian on-chain là `block.timestamp` (giây).
- Địa chỉ ví lưu và so sánh ở dạng chữ thường; chỉ định dạng checksum khi hiển thị.

2. Mục tiêu, ràng buộc và quyết định kiến trúc

2.1. Động lực thiết kế

Động lực	Nguồn	Hệ quả thiết kế
Không lưu khóa người dùng	BR-01, G-04	Mọi giao dịch và chữ ký của Buyer/Merchant sinh ra ở ví; backend chỉ xác minh, không bao giờ ký thay. Khóa sàn nằm sau một cổng ký (SignerPort) nạp từ môi trường.
Blockchain là nguồn sự thật	BR-09, G-05	Trạng thái đơn trong MySQL chỉ do Indexer ghi; giao diện không tin phản hồi của ví.
Giao hàng trong 10 tuần với 3 người	BRD mục 9	Ít thành phần di chuyển; dùng thư viện chuẩn (OpenZeppelin, eth_account, ethers.js); mock API cho frontend từ tuần 1.
Không xâm lấn lõi Slopee	SRS mục 2	Mô-đun Web3 là gói riêng, chỉ chạm lõi qua vài điểm nổi (tạo đơn, danh mục sản phẩm, vai trò người dùng).
Lặp lại được, kiểm thử được	G-06, AC-07, AC-08, AC-11	Mọi xử lý sự kiện idempotent; có replay; thời gian tách thành ClockPort và tua được trên Anvil.

2.2. Ràng buộc kỹ thuật

Solidity 0.8.20 + OpenZeppelin v5.0.2 (Foundry, `via_ir`); Python 3.11+ với Flask, web3.py, eth_account; MySQL 8.0 (InnoDB, utf8mb4); React + ethers.js v6 + MetaMask; chuỗi Anvil chainId 31337; token mặc định MockUSD.

2.3. Các quyết định thiết kế chính

Mã	Quyết định	Lý do	Đánh đổi
DD-01	Kiến trúc lục giác (Hexagonal) cho mô-đun Web3: miền nghiệp vụ không import Flask hay web3.py, giao tiếp qua cổng (port).	Test miền bằng bộ nhớ, không cần chuỗi; thay Anvil bằng testnet hoặc KMS ở Phase 2 chỉ đổi adapter.	Thêm một lớp trừu tượng, tốn khoảng 2-3 ngày dựng khung.
DD-02	Một contract duy nhất <code>SlopeeEscrowMaster</code> , không proxy, không pause.	Bề mặt tấn công nhỏ; đúng phạm vi BRD mục 2.1.	Sửa lỗi phải triển khai lại (chấp nhận được trên Anvil); đã chốt contract cuối tuần 2.
DD-03	Đơn chỉ được ký quỹ khi có báo giá EIP-712 do sàn ký; contract không có hàm tạo đơn riêng.	Chống chiếm mã đơn, tự đổi ví Merchant, nạp sai số tiền (BR-06).	Backend là điểm tin cậy cho giá và nhóm hàng; được giảm nhẹ bằng đổi chiều của Indexer.
DD-04	Giao diện đọc trạng thái đơn bằng polling <code>GET /api/orders/:code/status</code> từ MySQL, không đọc trực tiếp contract.	Một nguồn hiển thị duy nhất, khớp BR-09; đơn giản hơn WebSocket.	Độ trễ tối đa bằng chu kỳ polling cộng chu kỳ Indexer (mục tiêu dưới 10 giây).
DD-05	Indexer là tiến trình riêng, không chạy trong worker của web server.	Khởi động lại độc lập, không bị ảnh hưởng khi web server scale hoặc reload; dễ giữ một thể hiện duy nhất.	Thêm một tiến trình cần giám sát.

Mã	Quyết định	Lý do	Đánh đổi
DD-06	Relayer dùng một tài khoản gửi giao dịch, tuần tự hóa nonce qua hàng đợi một luồng.	Tránh lỗi nonce khi nhiều Shipper bấm cùng lúc.	Thông lượng thấp, đủ cho MVP.
DD-07	Số tiền lưu <code>VARCHAR(78)</code> và xử lý bằng <code>int / Decimal</code> ở Python, <code>BigInt</code> ở JavaScript.	<code>uint256</code> vượt giới hạn số nguyên 64-bit và <code>Number</code> .	Phải ép kiểu tương minh ở biên API (chuỗi trong JSON).
DD-08	Một tệp định nghĩa kiểu EIP-712 dùng chung (<code>shared/eip712/types.json</code>) cho Python, JS và test Solidity.	Lỗi lệch thứ tự trường là lỗi phổ biến nhất khi tích hợp.	Cần bước sinh/đọc tệp ở 3 nơi.
DD-09	Cổng ký <code>SignerPort</code> với adapter <code>EnvKeySigner</code> (MVP) và chỗ cắm KMS/HSM (Phase 2).	Giữ ngoại lệ BR-01 có kiểm soát; mỗi vai trò một khóa.	Không.
DD-10	MockUSD là phương thức mặc định; ETH chỉ có ở contract và test.	Tránh biến động giá khi demo (BRD mục 2).	Giao diện ETH là hạng mục nếu còn thời gian.

DD-11 (v1.2). Giỏ nhiều shop dùng mô hình thanh toán gom một lần, vận hành tách đơn độc lập: hàm `depositEscrowBatch` nguyên tử (tối đa 5 đơn, một token) dùng chung `_acceptQuote` với `depositEscrow`. Đánh đổi: một báo giá lỗi làm cả lô thất bại và phải cập lại cả lô; đổi lại Buyer chỉ xác nhận ví hai lần. Chi tiết ở mục 4.9.

DD-12 (v1.2). Toàn bộ Slopee dùng VND số nguyên (`DECIMAL(15, 0)`); không còn USD trên giao diện. Đổi lại phải chỉnh các cột tiền và khoảng 12 chỗ hiển thị ở frontend (16.3, bước 8).

3. Kiến trúc tổng thể

3.1. Ngữ cảnh hệ thống

Thực thể	Tương tác với mô-đun Web3
Buyer / Merchant	Dùng React; ký giao dịch và EIP-712 bằng MetaMask; gọi REST.

Thực thể	Tương tác với mô-đun Web3
Shipper	Dùng cổng Shipper; không có ví, relayer gửi giao dịch thay.
Admin	Dùng màn hình Admin; ký Trọng tài qua dịch vụ ký phía máy chủ.
Slopee Core	Cung cấp người dùng, sản phẩm, danh mục, bảng đơn hiện có. Xác thực và phân quyền chưa có, phải xây trước (mục 16).
Anvil EVM	Chạy contract và MockUSD; phát sự kiện cho Indexer.

3.2. Các tiến trình khi chạy

Tiến trình	Công nghệ	Trạng thái	Ghi chú
web	Flask (gunicorn 1-2 worker)	Phi trạng thái	REST API, chữ ký báo giá/phán quyết, relayer (qua hàng đợi nội bộ).
indexer	Python (web3.py), vòng lặp chu kỳ 1-2 giây	Con trỏ trong <code>indexer_cursor</code>	Đúng một thể hiện, bảo vệ bằng khóa <code>GET_LOCK('slopee_indexer')</code> của MySQL.
scheduler	Python (APScheduler hoặc cron)	Phi trạng thái	Quét báo giá hết hạn, tự gọi giải ngân đến hạn, nhắc hạn, hết hạn chữ ký phán quyết.
mysql	MySQL 8.0	Bền vững	Lỗi Slopee + bảng Web3.
anvil	Foundry Anvil	Tạm thời	Khởi tạo bằng một lệnh; mất trạng thái khi tắt trừ khi dùng <code>--state</code> .
frontend	React (Vite)	Tĩnh	Gọi REST và MetaMask.

3.3. Phân lớp Hexagonal của mô-đun Web3

```
slopee_web3/
  domain/          # thực thể, giá trị, quy tắc thuần (không I/O)
    money.py       # quy đổi VND -> token, tính phí, làm tròn
    policy.py      # chọn thời hạn theo nhóm hàng (lấy max)
    quote.py       # OrderQuote, chuẩn hóa docHash
    dispute.py     # máy trạng thái đề xuất phán quyết
    order_state.py # bảng chuyển trạng thái, hàm is_valid_transition
  application/     # ca sử dụng, điều phối các công
    wallet_service.py quote_service.py order_service.py
    shipper_service.py dispute_service.py settings_service.py
    reconcile_service.py
  ports/           # giao diện trừu tượng
    chain_port.py signer_port.py clock_port.py
    repositories.py notification_port.py
  adapters/
    web3_chain.py  # ChainPort qua web3.py
    env_signer.py  # SignerPort đọc khóa từ biến môi trường
    mysql_repos.py # repositories qua SQL tham số hóa
    flask_api/     # blueprint, schema, ánh xạ lỗi
    indexer/       # vòng lặp, handler từng sự kiện
    jobs/          # tác vụ nền
```

Quy tắc phụ thuộc: `domain` không import gì ngoài thư viện chuẩn; `application` chỉ phụ thuộc `domain` và `ports`; `adapters` phụ thuộc vào cả ba. Vi phạm bị chặn bằng một test kiểm tra import (T-ARCH-01).

3.4. Cấu trúc kho mã

```
repo/
  contracts/      # Foundry: src/, test/, script/, foundry.toml
  backend/        # slopee_web3 + điểm nối vào lõi Flask hiện có, migrations/
  frontend/       # React: pages/, hooks/, lib/
  shared/eip712/  # types.json, vectors.json (vector kiểm thử chéo ngôn ngữ)
  scripts/        # bootstrap.sh, reconcile.py, replay.sh
  docs/           # BRD, SRS, SDD, hướng dẫn cài đặt và demo
```

4. Thiết kế Smart Contract

4.1. Thành phần

Tệp	Nội dung
<code>src/SlopeeEscrowMaster.sol</code>	Contract chính (SRS mục 10), kế thừa <code>AccessControl</code> , <code>ReentrancyGuard</code> , <code>EIP712</code> .
<code>src/MockUSD.sol</code>	ERC-20 giả lập, có hàm <code>mint</code> cho môi trường thử; 6 chữ số thập phân (đã chốt Q-01).
<code>script/Deploy.s.sol</code>	Triển khai MockUSD và Escrow, truyền 5 địa chỉ (admin, shipper, arbitrator, quoteSigner, feeRecipient), in địa chỉ ra tệp JSON cho backend và frontend.
<code>test/*.t.sol</code>	Test đơn vị, fuzz, invariant (mục 13).

4.2. Thiết kế lưu trữ

Biến	Kiểu	Vai trò
<code>orders</code>	<code>mapping(uint256 => Order)</code>	Dữ liệu đơn; <code>state == NONE</code> nghĩa là mã đơn chưa dùng.
<code>disputeNonces</code>	<code>mapping(uint256 => uint256)</code>	Chống dùng lại chữ ký phán quyết theo từng đơn.
<code>pendingWithdrawals</code>	<code>mapping(token => mapping(addr => uint256))</code>	Sổ chờ rút (pull-payment).
<code>defaultFeeBps</code> , <code>feeRecipient</code>	<code>uint256</code> , <code>address</code>	Trần phí on-chain do Admin đặt; phí của đơn lấy từ báo giá (không vượt trần) và chốt vào <code>Order.feeBps</code> khi nạp tiền.

Struct `Order` không đóng gói (packing) biến để giữ mã dễ đọc và dễ kiểm toán; chi phí gas chấp nhận được với môi trường Anvil. Nếu cần tối ưu ở Phase 2, các trường thời gian có thể thu về `uint64` .

4.3. Thiết kế từng hàm

Hàm	Ai gọi	Điều kiện trước	Tác động	Sự kiện
<code>depositEscrow(q, sig)</code>	Buyer (<code>msg.sender</code>)	State NONE; chưa quá <code>deadline</code> ; amount lớn hơn 0; merchant hợp lệ và khác người nạp; hai thời hạn trong khoảng cho phép; Trọng tài không phải Buyer hay Merchant; người nạp trùng buyer trong báo giá; <code>feeBps</code> không vượt <code>defaultFeeBps</code> ; chữ ký báo giá của <code>QUOTE_SIGNER_ROLE</code>	Nhận ETH hoặc token (kiểm tra số dư trước/sau để loại token phí-khi-chuyển); ghi đơn; state LOCKED; chốt <code>feeBps = q.feeBps</code> (theo báo giá)	<code>OrderCreated</code>
<code>confirmDelivery(id)</code>	<code>SHIPPER_ROLE</code>	LOCKED	<code>deliveredAt = now</code> ; DELIVERED	<code>OrderDelivered</code>
<code>earlyRelease(id)</code>	Buyer	DELIVERED	<code>_executePayout</code>	<code>OrderCompleted</code>
<code>releaseAfterInspection(id)</code>	Bất kỳ	DELIVERED và <code>now >= deliveredAt + inspectionDuration</code>	<code>_executePayout</code>	<code>OrderCompleted</code>
<code>raiseDispute(id)</code>	Buyer	DELIVERED và <code>now < deliveredAt + inspectionDuration</code>	<code>disputedAt = now</code> ; DISPUTED	<code>OrderDisputed</code>
<code>resolveDispute(id, payoutTo, deadline, s1, s2)</code>	Bất kỳ	DISPUTED; <code>payoutTo</code> là Buyer hoặc Merchant; <code>now <= deadline</code> ; <code>deadline < disputedAt + disputeTimeout</code> ; hai chữ ký của hai bên khác nhau	Tăng nonce; <code>_settle</code>	<code>OrderRefunded</code> hoặc <code>OrderCompleted</code> , rồi <code>DisputeResolved</code>
<code>arbitratorForceResolve(id, payoutTo)</code>	<code>ARBITRATOR_ROLE</code>	DISPUTED và <code>now >= disputedAt + disputeTimeout</code> của đơn	<code>_settle</code>	<code>ArbitratorForceResolved</code>
<code>cancelIfUnfulfilled(id)</code>	Buyer hoặc Admin	LOCKED và <code>now >= createdAt + 14 ngày</code>	<code>_refund</code>	<code>OrderRefunded</code>
<code>claimPending(token)</code>	Chủ khoản chờ rút	Số dư chờ lớn hơn 0	Xóa số dư rồi mới chuyển	<code>PendingClaimed</code>
<code>setDefaultFeeBps, setFeeRecipient</code>	Admin	Phí tối đa 1.000 BPS; địa chỉ khác 0	Cập nhật tham số	<code>FeeUpdated</code> , <code>FeeRecipientUpdated</code>

Biên thời gian không chồng lấn: `raiseDispute` yêu cầu `now < hạn`, `releaseAfterInspection` yêu cầu `now >= hạn`, nên tại mỗi giây chỉ đúng một trong hai hàm hợp lệ. Đây là điểm cần có test biên (T-07, T-10).

Tại biên `disputedAt + disputeTimeout`, `resolveDispute` chỉ nhận chữ ký có `deadline` nhỏ hơn mốc này, còn `arbitratorForceResolve` hợp lệ từ đúng mốc đó. Nhờ vậy hai đường xử lý không bao giờ cùng hợp lệ ở một thời điểm (R-02).

4.4. Mẫu bảo vệ áp dụng

- **Checks-Effects-Interactions:** `_refund` và `_executePayout` đổi trạng thái trước khi chuyển tiền; `claimPending` xóa sổ dư trước khi gửi.
- **nonReentrant** trên mọi hàm di chuyển tiền; `confirmDelivery` và `raiseDispute` không chuyển tiền nên không cần.
- **Pull-payment** cho ETH: gọi `call` với giới hạn gas 30.000, thất bại thì ghi `pendingWithdrawals` thay vì revert, nhờ đó ví nhận độc hại không làm kẹt đơn.
- **SafeERC20** và kiểm tra chênh lệch sổ dư khi nạp token; token phí-khi-chuyển bị từ chối.
- **Phân tách vai trò:** Trọng tài không được là Buyer/Merchant; `_partyOf` ưu tiên vai Buyer/Merchant nếu một địa chỉ mang nhiều vai.
- **Không dùng `tx.origin`, không dùng `block.number` để đo thời gian;** mọi hạn dùng `block.timestamp`.

4.5. Thiết kế chữ ký

- Digest báo giá: `hashTypedDataV4(keccak256(abi.encode(QUOTE_TYPEHASH, orderId, buyer, merchant, token, amount, inspectionDuration, disputeTimeout, feeBps, docHash, deadline))))`. Struct `Quote` trong ABI và kiểu EIP-712 `OrderQuote` có cùng danh sách và thứ tự trường (đã thêm `buyer` và `feeBps`, xem R-01, R-05). Contract kiểm `q.buyer == msg.sender` và `q.feeBps <= defaultFeeBps`.
- Digest phán quyết: gồm `orderId`, `payoutTo`, `order.amount`, `disputeNonces[orderId]`, `deadline`. Số tiền lấy từ trạng thái đơn, nên chữ ký cho số tiền sai sẽ khôi phục ra địa chỉ lạ và bị loại. Hạn chữ ký phải nhỏ hơn `disputedAt + disputeTimeout`, do contract ép (R-02).
- Mỗi chữ ký được quy về một bên (1 Buyer, 2 Merchant, 3 Arbitrator); yêu cầu hai bên khác nhau và khác 0. Nonce tăng trước khi kiểm tra bên, nếu revert thì tăng bị hoàn tác.
- ECDSA của OpenZeppelin từ chối chữ ký malleable (s cao); MetaMask và `eth_account` tạo chữ ký s thấp nên tương thích.

4.6. Bất biến cần chứng minh bằng test

Mã	Bất biến
INV-01	Số dư contract theo từng token = tổng <code>amount</code> các đơn LOCKED, DELIVERED, DISPUTED + tổng số chờ rút.
INV-02	COMPLETED và REFUNDED không bao giờ chuyển sang trạng thái khác.
INV-03	Với đơn COMPLETED: <code>netPayout + feeAmount = amount</code> , và <code>feeAmount = floor(amount * feeBps / 10000)</code> .
INV-04	<code>disputeNonces[id]</code> không bao giờ giảm.
INV-05	Mỗi <code>orderId</code> chỉ nạp tiền thành công đúng một lần.
INV-06	<code>feeBps</code> của đơn bằng phí trong báo giá và không đổi sau khi tạo, dù <code>defaultFeeBps</code> đổi.

4.7. Triển khai và cấp vai trò

Trình tự trong `Deploy.s.sol` và `bootstrap.sh`: (1) khởi động Anvil với `--block-time 2`; (2) triển khai MockUSD; (3) triển khai Escrow với 5 địa chỉ khác nhau; (4) mint MockUSD cho tài khoản thử của Buyer; (5) ghi `deployments/local.json` (địa chỉ, chainId, block triển khai) để backend, indexer và frontend cùng đọc; (6) khởi tạo `indexer_cursor.last_block` bằng block triển khai trừ 1.

4.8. Thay đổi so với mã tham chiếu SRS v5.1

Sau rà soát (mục 15), mã tham chiếu ở SRS mục 10 được chỉnh ba điểm. Đoạn dưới là phần thay thế; các phần còn lại giữ nguyên.

```
bytes32 public constant QUOTE_TYPEHASH = keccak256(
    "OrderQuote(uint256 orderId,address buyer,address merchant,address
token,uint256 amount,uint256 inspectionDuration,uint256
disputeTimeout,uint256 feeBps,bytes32 docHash,uint256 deadline)"
);

struct Quote {
    uint256 orderId;
```

```

    address buyer;                // khớp 1-1 với kiểu EIP-712
    address merchant;
    address token;
    uint256 amount;
    uint256 inspectionDuration;
    uint256 disputeTimeout;
    uint256 feeBps;                // phí chốt trong báo giá
    bytes32 docHash;
    uint256 deadline;
}

// depositEscrow: thay phần kiểm tra và ghi phí
require(q.buyer == msg.sender, "Caller must be quote buyer");
require(q.feeBps <= defaultFeeBps, "Fee exceeds on-chain cap");
// _verifyQuote bấm đúng thứ tự trường của QUOTE_TYPEHASH (dùng q.buyer thay
cho msg.sender)
o.feeBps = q.feeBps;

// resolveDispute: thêm sau kiểm tra deadline
require(_deadline < order.disputedAt + order.disputeTimeout, "Deadline must
be before dispute timeout");

```

Hệ quả: `defaultFeeBps` trở thành trần phí do Admin đặt trên chuỗi (vẫn tối đa 1.000 BPS), phí thực của từng đơn do báo giá quyết định. Bổ sung bất biến **INV-07**: với mọi đơn, `feeBps` bằng giá trị trong báo giá đã ký và không vượt `defaultFeeBps` tại thời điểm nạp.

4.9. Nạp gom nhiều shop (depositEscrowBatch)

Giỏ hàng có nhiều shop vẫn thanh toán bằng đúng hai lần xác nhận ví: `approve` tổng số tiền của cả giỏ, rồi một giao dịch `depositEscrowBatch`. Sàn tách giỏ thành một đơn cho mỗi shop; mỗi đơn có báo giá, `orderId`, thời hạn và Merchant riêng, và sau khi nạp vận hành hoàn toàn độc lập (BR-11). Mã tham chiếu ở SRS mục 10; mục này nêu thiết kế và các điểm đã chỉnh so với đề xuất ban đầu.

```

depositEscrow(q, sig)                -> _acceptQuote(q, sig); _collect(q.token,
q.amount)
depositEscrowBatch(quotes, sigs) -> mỗi báo giá: _acceptQuote(...), cộng dồn
tổng
                                cuối cùng: _collect(token, tổng)
_acceptQuote: mọi kiểm tra của một báo giá, ghi đơn LOCKED, emit OrderCreated
(không chuyển tiền)
_collect:      thu ETH hoặc token đúng một lần

```

Vấn đề trong đề xuất ban đầu	Hậu quả nếu giữ	Cách xử lý
Chỉ kiểm vài điều kiện, bỏ sót các kiểm tra của <code>depositEscrow</code> (số tiền khác 0, Merchant hợp lệ và khác người nạp, khoảng thời hạn kiểm tra và tranh chấp, Trọng tài không được giao dịch)	Đường nạp gom yếu hơn đường nạp đơn: tạo được đơn số tiền 0, thời hạn ngoài khoảng, Trọng tài làm Merchant	Dùng chung <code>_acceptQuote</code> cho cả hai hàm
Kiểm tra <code>state == NONE</code> cho cả lô trước, ghi đơn sau	Hai báo giá cùng <code>orderId</code> đều qua kiểm tra; tiền bị thu hai lần nhưng chỉ ghi một đơn, tiền kẹt và sai INV-01	Ghi đơn LOCKED ngay trong vòng lặp; báo giá trùng bị từ chối <code>Order exists</code>
Không giới hạn số phần tử	Hết gas, giao dịch không thể vào block	<code>MAX_BATCH_SIZE = 5</code> , đo gas ở T-32; backend trả <code>BATCH_TOO_LARGE</code> khi giở vượt
Hai nhánh nạp ETH và token lặp mã ở hai hàm	Hai hàm có thể lệch hành vi theo thời gian	<code>_collect</code> dùng chung

Tính chất. Nguyên tử: một báo giá lỗi, hết hạn, phí vượt trần hay sai người nạp làm cả lô revert, nên mọi báo giá của một lô có cùng `deadline` và backend cập lại báo giá cho cả lô (5.3f). Độc lập: sau khi nạp, mọi hàm thao tác trên một `orderId`, không hàm nào chạm đơn khác; INV-01 đến INV-07 vẫn đúng. Tổng tiền là tổng `amount` từng đơn (mỗi đơn đã làm tròn lên riêng), một token cho cả lô.

Indexer không đổi. Một lần nạp gom phát N sự kiện `OrderCreated` cùng `tx_hash`, khác `log_index`; Indexer xử lý từng sự kiện như trước, nên không cần logic lô riêng. Nhóm các đơn trên giao diện bằng `checkout_group_id` do backend lưu.

5. Thiết kế Backend (Flask)

5.1. Các mô-đun

Mã	Mô-đun	Trách nhiệm	Phụ thuộc (cổng)
MOD-01	WalletService	Cấp nonce, xác minh chữ ký liên kết ví, đổi ví (UC-00)	Repos, Clock
MOD-02	PolicyService	Tra <code>category_policies</code> , chọn thời hạn, hiển thị chính sách	Repos
MOD-03	QuoteService	Tạo đơn PENDING_PAYMENT, quy đổi tiền, tính docHash, ký và lưu báo giá (UC-01)	Repos, Signer, Chain, Clock
MOD-04	OrderQueryService	Chi tiết và trạng thái đơn cho polling	Repos
MOD-05	ShipperService	Kiểm quyền và gửi <code>confirmDelivery</code> qua relay (UC-03)	Repos, Chain, Notification
MOD-06	DisputeService	Đề xuất phán quyết, xác minh và lưu chữ ký, nộp <code>resolveDispute</code> (UC-06)	Repos, Signer, Chain, Clock
MOD-07	SettingsService	Tham số vận hành, chính sách nhóm hàng, ghi người sửa (UC-09)	Repos
MOD-08	ReconcileService	Đối soát sổ dư contract với đơn đang mở (AC-10)	Repos, Chain
MOD-09	Indexer	Đồng bộ sự kiện (mục 6)	Repos, Chain
MOD-10	Jobs	Tác vụ nền (mục 5.5)	Các service trên

5.2. Cổng (ports)

Cổng	Phương thức chính	Adapter MVP	Phase 2
ChainPort	<code>get_order(id)</code> , <code>get_logs(from, to)</code> , <code>get_block(n)</code> , <code>send_tx(fn, args, role)</code> , <code>balance_of(token)</code> , <code>dispute_nonce(id)</code>	web3.py vào Anvil	RPC testnet
SignerPort	<code>sign_quote(quote)</code> , <code>sign_dispute(res)</code> , <code>sender_address(role)</code>	EnvKeySigner	KMS/HSM, multisig
ClockPort	<code>now_system()</code> , <code>now_chain()</code>	HybridClock (đặt ở adapters/): <code>now_chain = max(đồng hồ hệ thống, thời gian block mới nhất)</code> ; đồng hồ giả trong test	-
Repositories	Mỗi bảng một repository, giao dịch qua <code>UnitOfWork</code>	MySQL (<code>mysql-connector</code> hoặc <code>SQLAlchemy Core</code>)	-
NotificationPort	<code>notify_buyer_delivered(order)</code>	Ghi thông báo vào bảng thông báo của Slopee	Email/push

`SignerPort` chỉ nhận dữ liệu có cấu trúc đã kiểm hợp lệ và trả chữ ký; không có phương thức trả khóa ra ngoài. Không mô-đun nào khác được đọc biến môi trường chứa khóa.

Quy tắc đồng hồ (R-09). `now_system()` dùng cho nonce liên kết ví, phiên đăng nhập, giới hạn tần suất. `now_chain()` bằng `max(đồng hồ hệ thống, thời gian block mới nhất)`, dùng cho hạn báo giá, hạn chữ ký phán quyết, tác vụ nền đếm hạn và `server_time` trả cho frontend. Lớp này hiện thực ở `adapters/` vì phụ thuộc web3. Anvil chạy với `--block-time 2` để block mới nhất luôn gần thời gian chuỗi hiện tại; chỉ tua thời gian trong kịch bản kiểm thử và cập báo giá mới sau mỗi lần tua.

5.3. Thuật toán cốt lõi

a) Quy đổi VND sang token (BR-08). Tỷ giá `FX_VND_PER_TOKEN` lưu `DECIMAL(18,4)`, đọc thành số nguyên nhân 10^4 (`fx_scaled`) để tránh số thực:

```
amount_raw = ceil( price_vnd * 10^decimals * 10^4 / fx_scaled )
              = (price_vnd * 10^decimals * 10^4 + fx_scaled - 1) // fx_scaled
```

Làm tròn lên có nghĩa sàn không thiệt. Tỷ giá và số token được lưu vào `orders` (`fx_vnd_per_token`, `amount_raw`) và báo giá.

Với giỏ nhiều shop, tổng của lô là tổng `amount_raw` đã làm tròn lên của từng shop, không làm tròn lại trên tổng, để khớp đúng số tiền contract thu. Giá và tổng tiền VND của mỗi shop phải là số nguyên (toàn Slopee dùng VND, DD-12).

b) Chọn thời hạn theo nhóm hàng (BR-10). Với mỗi sản phẩm trong đơn lấy `category_code`; không xác định được thì dùng nhóm `is_default = 1`. `inspectionDuration = max(...)` và `disputeTimeout = max(...)` riêng biệt. Sau đó kiểm lại hai giới hạn 1 giờ-30 ngày và 7-90 ngày ở backend trước khi ký, để lỗi cấu hình được phát hiện sớm thay vì bị contract revert.

c) docHash. `docHash = keccak256(canonical_json(summary))`, trong đó `summary` gồm `order_code`, danh sách hàng (mã sản phẩm, số lượng, đơn giá VND), `price_vnd`, thời điểm tạo (giây UTC, số nguyên). Chuẩn hóa: khóa sắp xếp tăng dần, không khoảng trắng thừa, UTF-8, chỉ dùng số nguyên. Vì docHash là bằng chứng sau này, hàm chuẩn hóa có vector kiểm thử cố định (mục 8.3).

d) Cấp `onchain_order_id`. Dùng một bảng đếm riêng (`AUTO_INCREMENT`, khởi đầu từ giá trị cấu hình) để số đơn tăng đơn điệu, không dùng lại kể cả khi đơn bị xóa. Giữ giá trị nhỏ hơn 2^{53} để an toàn khi hiển thị ở JavaScript.

e) Phát hành báo giá (UC-01), trong một giao dịch CSDL:

1. Kiểm tra Buyer và mọi Merchant của giỏ có ví `is_active = 1` (BR-07); số shop không quá `MAX_BATCH_SIZE` (5), nếu vượt trả `BATCH_TOO_LARGE`; sản phẩm bắt thanh toán Web3; tổng tiền của mỗi shop là số nguyên VND.
2. Nhóm mặt hàng theo shop. Với mỗi shop: cấp `onchain_order_id` riêng, tính `amount_raw` (làm tròn lên riêng), chọn thời hạn theo nhóm hàng của shop đó, tính `docHash` riêng. Đọc `PLATFORM_FEE_BPS` và trần `defaultFeeBps` trên chuỗi, đặt `feeBps = min(cấu hình, trần)` (R-01).
3. Gọi `SignerPort.sign_quote` cho từng shop, tất cả cùng `deadline = now_chain + QUOTE_TTL_SECONDS`.

4. Trong một giao dịch CSDL: ghi `checkout_groups` ; với mỗi shop ghi `orders` (pending), `order_web3` (cùng `checkout_group_id`) và `order_quotes` ; giữ chỗ tồn kho. Lỗi bất kỳ thì rollback cả lô. Trả `checkout_group_id` , `total_amount_raw` (tổng `amount_raw`) và mảng đơn kèm báo giá.

f) Cập lại báo giá. Chỉ giữ nguyên `onchain_order_id` khi

`ChainPort.get_order(id).state == NONE` (kiểm tra trực tiếp trên chuỗi, không chỉ dựa vào CSDL, vì Indexer có thể đang trễ). Nếu chuỗi đã có đơn thì từ chối cập lại và để Indexer cập nhật.

Cập lại báo giá cho lô. Vì nạp gom là nguyên tử, báo giá được cập lại cho cả lô: giữ nguyên các `onchain_order_id` khi contract trả `NONE` cho từng mã, đặt `deadline` mới chung và đọc lại trần phí. Khi lô hết hạn mà chưa nạp, cả lô chuyển EXPIRED cùng lúc và hoàn tồn kho.

g) Liên kết ví (UC-00). Thông điệp theo mẫu SIWE: tên miền, địa chỉ, mã tài khoản, nonce, chainId, thời điểm phát hành và hết hạn. Tiêu thụ nonce nguyên tử để chống dùng lại khi có hai yêu cầu song song:

```
UPDATE wallet_nonces SET used_at = NOW()
WHERE nonce = :n AND user_id = :u AND used_at IS NULL AND expires_at >
NOW();
-- rowcount = 1 thì hợp lệ; ngược lại trả NONCE_INVALID
```

Sau đó khôi phục địa chỉ bằng `Account.recover_message` , so với địa chỉ đã gửi (chữ thường), rồi chèn `user_wallets` . Ràng buộc `UNIQUE(address)` xử lý trường hợp ví đã thuộc tài khoản khác.

h) Relayer cho Shipper (UC-03). Trước khi gửi, đọc `get_order(id)` : nếu đã DELIVERED thì trả thành công (idempotent), nếu không phải LOCKED thì trả `INVALID_STATE` . Gửi giao dịch qua hàng đợi một luồng (DD-06), chờ biên nhận, ghi bảng `tx_outbox` . Relayer **không tự cập nhật trạng thái đơn**; việc đó do Indexer làm (BR-09).

i) Phán quyết (UC-06). Máy trạng thái đề xuất:

Từ	Sự kiện	Sang
(mới)	Admin tạo đề xuất, Trọng tài ký	PROPOSED
PROPOSED	Một bên Buyer hoặc Merchant nộp chữ ký hợp lệ	READY

Từ	Sự kiện	Sang
READY	Relayer hoặc bên được lợi nộp <code>resolveDispute</code>	SUBMITTED
SUBMITTED	Indexer thấy <code>DisputeResolved</code>	EXECUTED
PROPOSED, READY	<code>deadline</code> qua hoặc Admin tạo đề xuất mới	EXPIRED / CANCELLED

Vì chữ ký Trọng tài đã có ngay khi tạo, chỉ cần **một** chữ ký của Buyer hoặc Merchant là đủ 2/3. Do đó giao diện phải nêu rõ cho bên ký rằng việc ký của họ là cái kích hoạt thi hành. Mỗi đơn chỉ có một đề xuất đang hoạt động, kiểm tra bằng `SELECT ... FOR UPDATE` trên dòng đơn trước khi tạo. `deadline = min(now_chain + DISPUTE_SIG_TTL_SECONDS, disputedAt + disputeTimeout - 1); nonce` đọc từ `disputeNonces(orderId)` ngay lúc tạo. Khi xác minh chữ ký Buyer/Merchant, dùng `encode_typed_data` từ cùng tệp kiểu chung (DD-08), khôi phục địa chỉ và so với ví `is_active` của bên đó.

5.4. Mô hình lỗi

Ngoại lệ miền được ánh xạ ở biên API thành HTTP 4xx kèm `code` và thông báo tiếng Việt (SRS mục 7).

Ngoại lệ miền	HTTP	code
Chưa liên kết ví	409	WALLET_NOT_LINKED
Ví thuộc tài khoản khác	409	WALLET_TAKEN
Nonce hết hạn hay đã dùng	400	NONCE_INVALID
Chữ ký sai hoặc không khớp ví	400	BAD_SIGNATURE
Báo giá hết hạn	410	QUOTE_EXPIRED
Đơn sai trạng thái cho thao tác	409	INVALID_STATE
Không có quyền trên đơn	403	FORBIDDEN
Chính sách ngoài giới hạn	422	POLICY_OUT_OF_RANGE
Giao dịch chuỗi thất bại	502	CHAIN_TX_FAILED (kèm lý do revert đã rút gọn)

Mã lỗi bổ sung: `FEE_CAP_CHANGED` (409) khi Admin hạ trần phí on-chain sau khi báo giá được phát hành, làm báo giá cũ không còn nạp được; frontend gọi `POST /api/orders/:code/quote` để lấy báo giá mới. Quy trình đổi phí (UC-09): Admin gọi `setDefaultFeeBps` trên chuỗi trước, rồi mới đổi `PLATFORM_FEE_BPS` khi tăng phí; khi hạ phí thì đổi cấu hình trước để báo giá mới đã thấp hơn.

Mã lỗi `BATCH_T00_LARGE` (422) khi giỏ có nhiều hơn 5 shop; giao diện yêu cầu bỏ bớt sản phẩm. Lỗi revert `Invalid batch size` của contract được ánh xạ về cùng thông báo.

5.5. Tác vụ nền

Tác vụ	Chu kỳ	Hành động	Tính idempotent
QuoteExpirySweeper	1 phút	Đơn <code>PENDING_PAYMENT</code> có báo giá mới nhất quá hạn và chuỗi chưa có đơn → <code>EXPIRED</code>	Có điều kiện trạng thái trong câu <code>UPDATE</code>
AutoRelease	1 phút	Đơn <code>DELIVERED</code> đến hạn → gọi <code>releaseAfterInspection</code> qua relayer	Kiểm trạng thái trên chuỗi trước; revert do cạnh tranh được bỏ qua
DeliveryTimeoutReminder	1 giờ	Nhắc Buyer có đơn <code>LOCKED</code> đến mốc 14 ngày	Đánh dấu đã nhắc
DisputeSigExpiry	5 phút	Đề xuất <code>PROPOSED/READY</code> quá <code>deadline</code> → <code>EXPIRED</code>	Có điều kiện trạng thái
ReconcileSnapshot	10 phút	Chạy đối soát, ghi kết quả cho Admin	Chỉ đọc

5.6. Cấu hình và nhật ký

Khóa đọc một lần lúc khởi động vào `EnvKeySigner`, kiểm 4 địa chỉ suy ra phải khác nhau, và địa chỉ khớp với vai trò đã cấp trên contract; sai là dừng khởi động. Nhật ký dùng bộ lọc che mây trường `signature`, `private`, `key`, `mnemonic`; test E-05 quét cả mã nguồn, CSDL và log.

6. Thiết kế Indexer

6.1. Vòng lặp xử lý

```

acquire GET_LOCK('slopee_indexer')          # chỉ một thể'hiện
loop mỗi POLL_SECONDS:
    cursor = load indexer_cursor('escrow')
    if cursor.last_block_hash != chain.block(cursor.last_block).hash:
        cursor.last_block -= REORG_DEPTH (mặc định 20); rollback dữ liệu dẫn
xuất từ block đó
    to_block = chain.latest - CONFIRMATIONS
    for each window (tối đa 500 block) from cursor.last_block+1 to to_block:
        logs = chain.get_logs(window) sorted by (block, logIndex)
        BEGIN
            for log in logs:
                if chain_events has (tx_hash, log_index): continue
                insert chain_events(..., block_timestamp)
                handler[log.event](log)
            update indexer_cursor(last_block=window.end, last_block_hash)
        COMMIT          # lỗi bất kỳ → ROLLBACK, lần sau làm lại từ đầu cửa sổ'

```

Mỗi cửa sổ là một giao dịch, nên con trỏ và dữ liệu nghiệp vụ luôn nhất quán. Độ trễ mục tiêu dưới 2 giây mỗi vòng trên Anvil.

Thời gian block (R-04). Mỗi log được gắn `block_timestamp`. Indexer giữ một bảng băm tạm theo `block_number` trong phạm vi cửa sổ đang xử lý, nên block có nhiều log chỉ gọi `eth_getBlockByNumber` đúng một lần. Các mốc `delivered_at`, `disputed_at`, `closed_at`, `chain_created_at` đều lấy từ giá trị này, không bao giờ từ `NOW()` của máy chủ, để replay cho kết quả giống hệt.

6.2. Bộ xử lý từng sự kiện

Mỗi handler chỉ áp dụng chuyển trạng thái **hợp lệ theo bảng SRS mục 5** (`is_valid_transition`). Chuyển không hợp lệ không được làm lùi trạng thái; nó đặt `anomaly_flag = 1` và ghi nhật ký cảnh báo.

Sự kiện	Ghi vào CSDL	Ghi chú thiết kế
OrderCreated	Tra <code>order_web3</code> theo <code>onchain_order_id</code> ; đối chiếu buyer, merchant, token, amount, inspectionDuration, disputeTimeout, feeBps, docHash với báo giá đã lưu; đặt LOCKED, <code>deposit_tx_hash</code> , <code>chain_created_at</code> = thời gian block	Lệch → <code>anomaly_flag</code> , không sửa dữ liệu sẵn; đối chiếu thêm với <code>category_policies</code> theo <code>category_code</code> . Phí đã chốt trong báo giá nên Indexer không ghi đề <code>fee_bps</code> .
OrderDelivered	DELIVERED, <code>delivered_at</code> theo tham số sự kiện	Chỉ từ LOCKED.
OrderDisputed	DISPUTED, <code>disputed_at</code>	Chỉ từ DELIVERED.
OrderCompleted	COMPLETED, <code>closed_at</code> = thời gian block	Từ DELIVERED hoặc DISPUTED.
OrderRefunded	REFUNDED, <code>closed_at</code>	Từ LOCKED hoặc DISPUTED.
DisputeResolved, ArbitratorForceResolved	Đề xuất tương ứng → EXECUTED, ghi cách giải quyết	Trong cùng giao dịch chuỗi, <code>OrderCompleted/Refunded</code> đứng trước <code>DisputeResolved</code> .
PayoutDeferred, PendingClaimed	Cập nhật <code>pending_balances</code>	Dùng cho giao diện rút và đối soát.
FeeUpdated, FeeRecipientUpdated	Chỉ <code>chain_events</code>	-

Mọi handler đổi `status` còn cập nhật bảng `orders` cũ trong cùng giao dịch CSDL (mục 7.4).

Với nạp gom, nhiều `OrderCreated` cùng `tx_hash` được xử lý lần lượt theo `log_index`. Khi một đơn chuyển LOCKED, handler xóa các `cartItems` của sản phẩm thuộc đơn đó khỏi giỏ của Buyer (cùng giao dịch CSDL), nên giỏ chỉ được dọn sau khi tiền đã khóa.

6.3. Đối soát, replay, phát hiện bất thường

- **Đối soát số dư (AC-10):** với mỗi token, $\Sigma \text{ amount_raw}$ của đơn LOCKED, DELIVERED, DISPUTED cộng $\Sigma \text{ PayoutDeferred} - \Sigma \text{ PendingClaimed}$ phải bằng $\text{balanceOf}(\text{contract})$ (ETH: eth_getBalance). Lệnh thì Admin thấy cảnh báo đỏ.
- **Replay (AC-08):** `--from-block 0 --reset` xóa `chain_events`, đặt con trỏ về 0 và đưa các cột dẫn xuất (`status`, `delivered_at`, `disputed_at`, `closed_at`, `deposit_tx_hash`) về giá trị ban đầu; các cột do sinh từ báo giá (giá VND, `doc_hash`, nhóm hàng) không bị đụng. Test so sánh ảnh chụp bằng `orders` trước và sau replay.
- **Sự kiện cho đơn không tồn tại ở sẵn:** chỉ lưu `chain_events` và cảnh báo (giả mạo hoặc nạp bằng đường khác).
- **Reorg:** chỉ có ý nghĩa trên mạng thật; trên Anvil `CONFIRMATIONS = 0`, cơ chế so sánh hash block vẫn để sẵn để nâng cấp.
- **Chịu lỗi:** mất kết nối RPC thì thử lại với backoff tăng dần tới 30 giây; không làm sập tiến trình. Phát hiện Anvil bị khởi động lại (block mới nhất nhỏ hơn con trỏ) thì dừng và báo cần chạy `bootstrap` + replay, thay vì chạy tiếp trên dữ liệu lệch.

7. Thiết kế dữ liệu

Lược đồ gốc nằm ở SRS mục 9. Theo quyết định Q-04, thay vì mở rộng bảng `orders` hiện có của Slopee, các cột Web3 nằm trong bảng riêng `order_web3` quan hệ 1-1 với `orders`; mọi chỗ SRS nói về cột Web3 của `orders` đọc thành `order_web3`. Mục này nêu quyền ghi, lược đồ bảng mới, các bổ sung và cách đồng bộ với bảng cũ.

7.1. Quyền ghi theo thành phần

Bảng / cột	Ai được ghi	Ghi chú
<code>order_web3</code> : thông tin báo giá (giá, tỷ giá, số token, nhóm hàng, hai thời hạn, <code>fee_bps</code> , <code>doc_hash</code>)	QuoteService	Bất biến sau khi LOCKED.
<code>order_web3.status</code> từ LOCKED trở đi; <code>delivered_at</code> , <code>disputed_at</code> , <code>closed_at</code> , <code>chain_created_at</code> , <code>deposit_tx_hash</code>	Chỉ Indexer	BR-09.

Bảng / cột	Ai được ghi	Ghi chú
<code>order_web3.status</code> ở PENDING_PAYMENT, EXPIRED	QuoteService, QuoteExpirySweeper	Chỉ có ở sàn.
<code>order_web3.anomaly_flag</code>	Indexer	Admin chỉ đọc.
<code>orders</code> (bảng Slopee cũ): cột trạng thái	Indexer và QuoteExpirySweeper, qua một hàm đồng bộ duy nhất	Chỉ là bản chiếu cho giao diện cũ, không phải nguồn sự thật.
<code>dispute_resolutions</code> , <code>dispute_signatures</code>	DisputeService (PROPOSED đến SUBMITTED); Indexer (EXECUTED)	-
<code>chain_events</code> , <code>indexer_cursor</code>	Indexer	-
<code>category_policies</code> , <code>platform_settings</code>	SettingsService (Admin)	Ghi <code>updated_by</code> .

7.2. Bảng order_web3

```

CREATE TABLE IF NOT EXISTS checkout_groups (
  group_id VARCHAR(36) PRIMARY KEY,
  buyer_user_id VARCHAR(36) NOT NULL,
  token_address CHAR(42) DEFAULT NULL,
  total_amount_raw VARCHAR(78) NOT NULL,          -- tổng amount_raw của
các đơn con
  order_count TINYINT UNSIGNED NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT chk_cg_count CHECK (order_count BETWEEN 1 AND 5),
  CONSTRAINT fk_cg_buyer FOREIGN KEY (buyer_user_id) REFERENCES users(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS order_web3 (
  order_id VARCHAR(36) NOT NULL PRIMARY KEY,      -- = orders.id (UUID);
cùng charset và collation với orders.id
  onchain_order_id BIGINT UNSIGNED NOT NULL UNIQUE, -- cấp từ
onchain_id_seq, bắt đầu 100001
  buyer_user_id VARCHAR(36) NOT NULL,            -- = users.id
  merchant_user_id VARCHAR(36) NOT NULL,          -- = shops.sellerId
  shop_id VARCHAR(36) NOT NULL,
  checkout_group_id VARCHAR(36) NOT NULL,         -- lô thanh toán chứa

```

```

đơn này
    buyer_wallet CHAR(42) NOT NULL,
    merchant_wallet CHAR(42) NOT NULL,
    token_address CHAR(42) DEFAULT NULL,          -- NULL = ETH, khác
NULL = ERC-20
    token_symbol VARCHAR(16) NOT NULL DEFAULT 'mUSD',
    amount_raw VARCHAR(78) NOT NULL,
    price_vnd BIGINT UNSIGNED NOT NULL,          -- từ
orders.totalAmount (VND số nguyên)
    fx_vnd_per_token DECIMAL(18,4) NOT NULL,
    payment_method VARCHAR(20) NOT NULL DEFAULT 'CRYPTO_ESCROW',
    status
ENUM('PENDING_PAYMENT', 'EXPIRED', 'LOCKED', 'DELIVERED', 'DISPUTED', 'COMPLETED',
'REFUNDED')
    NOT NULL DEFAULT 'PENDING_PAYMENT',
    policy_code VARCHAR(32) NOT NULL DEFAULT 'GENERAL',
    fee_bps INT UNSIGNED NOT NULL,              -- phí chốt trong báo
giá
    inspection_duration INT UNSIGNED NOT NULL,
    dispute_timeout INT UNSIGNED NOT NULL,
    doc_hash CHAR(66) NOT NULL,
    deposit_tx_hash CHAR(66) DEFAULT NULL,
    chain_created_at BIGINT UNSIGNED DEFAULT NULL, -- giây UTC theo thời
gian block
    delivered_at BIGINT UNSIGNED DEFAULT NULL,
    disputed_at BIGINT UNSIGNED DEFAULT NULL,
    closed_at BIGINT UNSIGNED DEFAULT NULL,
    anomaly_flag TINYINT(1) NOT NULL DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    INDEX idx_buyer (buyer_wallet),
    INDEX idx_merchant (merchant_wallet),
    INDEX idx_status (status),
    INDEX idx_group (checkout_group_id),
    CONSTRAINT chk_onchain_id CHECK (onchain_order_id < 9007199254740992),
    CONSTRAINT chk_fee CHECK (fee_bps <= 1000),
    CONSTRAINT fk_order_web3_group FOREIGN KEY (checkout_group_id) REFERENCES
checkout_groups(group_id),
    CONSTRAINT fk_order_web3_parent FOREIGN KEY (order_id) REFERENCES
orders(id) ON DELETE RESTRICT,
    CONSTRAINT fk_order_web3_buyer FOREIGN KEY (buyer_user_id) REFERENCES

```

```
users(id) ON DELETE RESTRICT,
    CONSTRAINT fk_order_web3_merchant FOREIGN KEY (merchant_user_id)
REFERENCES users(id) ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Sau khi đọc mã Slopee (mục 16), bảng đã chỉnh theo thực tế: `orders.id` và `users.id` là `VARCHAR(36)` (UUID) chứ không phải `INT`; mã đơn của sàn chính là `orders.id` nên bỏ cột `order_code`; thêm `shop_id`; `category_code` đổi thành `policy_code` tra qua bảng ánh xạ (7.3). Các mốc lấy từ thời gian block lưu `BIGINT` giây UTC; có `CHECK` chặn ID vượt 2^{53} (cần MySQL 8.0.16 trở lên).

Migration phòng vệ (Q-06). Tập 001 đọc `information_schema` cho `orders.id`: kiểu cột, charset, collation và engine của bảng. Nếu bảng không phải InnoDB thì dừng và báo rõ (MyISAM không hỗ trợ khóa ngoại); nếu khác `VARCHAR(36)` thì sinh cột `order_id` đúng kiểu đó. Charset và collation của `order_id`, `buyer_user_id`, `merchant_user_id` phải khớp cột cha, nếu không khóa ngoại lỗi errno 150. Khóa ngoại dùng `ON DELETE RESTRICT`. Lưu ý: `DELETE FROM users` của admin hiện tại sẽ bị chặn khi người dùng có đơn Web3, nên chức năng xóa người dùng phải xử lý lỗi này.

7.3. Các bổ sung khác

#	Bổ sung	Lý do
1	Cột <code>block_timestamp</code> <code>BIGINT UNSIGNED NOT NULL</code> ở <code>chain_events</code>	Nguồn thời gian cho mọi mốc dẫn xuất và cho replay (R-04).
2	Bảng <code>tx_outbox(id, kind, order_id, tx_hash, status, error, created_at)</code>	Truy vết giao dịch relayer, hỗ trợ gỡ lỗi demo.
3	Bảng <code>onchain_id_seq</code> với <code>AUTO_INCREMENT</code> bắt đầu từ 100001	ID đơn đơn điệu, nhỏ hơn 2^{53} rất nhiều (R-03). Frontend giữ ID dạng chuỗi hoặc <code>BigInt</code> .
4	Bảng dẫn xuất <code>pending_balances(token, address, amount_raw)</code>	Giao diện rút khoản chờ rút và đối soát.
5	Khóa <code>PLATFORM_FEE_BPS</code> (mặc định 300) trong <code>platform_settings</code>	Phí cấu hình của sàn; báo giá lấy <code>min(PLATFORM_FEE_BPS, defaultFeeBps on-chain)</code> (R-01).

#	Bổ sung	Lý do
6	Vai trò Shipper: thêm giá trị <code>shipper</code> vào cuối ENUM <code>users.role</code> và bảng <code>shipper_assignments(order_id, shipper_user_id)</code>	Slopee không có vai trò Shipper cũng không có vận đơn; thay cho giả định tái dùng bảng có sẵn ở R-08.
7	Bảng <code>category_policy_map(category_id INT UNSIGNED PRIMARY KEY, policy_code)</code>	<code>categories</code> chỉ có <code>id</code> , <code>name</code> . Dùng bảng ánh xạ để không phải ALTER bảng cũ; danh mục chưa ánh xạ dùng nhóm GENERAL. Màn hình quản lý danh mục của Admin cần thêm lựa chọn nhóm chính sách.
8	Đổi kiểu khóa của các bảng SRS mục 9 cho khớp Slopee	<code>user_wallets.user_id</code> , <code>wallet_nonces.user_id</code> , <code>dispute_resolutions.created_by_admin</code> , <code>platform_settings.updated_by</code> , <code>category_policies.updated_by</code> thành <code>VARCHAR(36)</code> ; <code>order_quotes.order_id</code> , <code>dispute_resolutions.order_id</code> thành <code>VARCHAR(36)</code> và tham chiếu <code>order_web3(order_id)</code> .

Bổ sung theo v1.2: bảng `checkout_groups` (một dòng cho mỗi lần thanh toán gom, trạng thái của lô suy ra từ các đơn con vì nạp gom là nguyên tử); cột `order_web3.checkout_group_id`; và đổi bốn cột tiền của Slopee sang `DECIMAL(15,0)` (`products.unitPrice`, `orderLines.unitPrice`, `orders.totalAmount`, `paymentMethods.balance`), dữ liệu cũ tính theo USD nhân với `FX_VND_PER_TOKEN` trước khi đổi kiểu.

7.4. Đồng bộ ngược sang bảng orders cũ

Một hàm duy nhất `sync_legacy_status(tx, order_id, web3_status)` được gọi trong cùng giao dịch CSDL với mọi thay đổi `order_web3.status`, để giao diện và báo cáo hiện có của Slopee không bị lệch.

Trạng thái Web3	<code>orders.status</code> cũ	Ghi chú
PENDING_PAYMENT	<code>pending</code>	Tạo đơn kèm báo giá.

Trạng thái Web3	orders.status cũ	Ghi chú
LOCKED	paid	Tiền đã khóa trong contract.
DELIVERED	shipped	-
DISPUTED	shipped	Giữ nguyên; giao diện hiện huy hiệu vàng Khiếu nại Web3 lấy từ order_web3 và khóa các nút thao tác thường.
COMPLETED	received	-
EXPIRED, REFUNDED	cancelled	Khôi phục tồn kho khi chuyển sang trạng thái này.

Giá trị ENUM đã lấy từ schema.sql của Slopee (pending, paid, shipped, cancelled, received), nên không cần ALTER orders.status (Q-07). Hai điểm chạm vào mã cũ, cả hai nhỏ: (1) GET /api/payments/orders/<user_id> thêm LEFT JOIN order_web3 để trả thêm cờ tranh chấp, đây là nơi duy nhất hiện liệt kê đơn; (2) PUT /api/payments/orders/<order_id> là đường ghi duy nhất vào orders.status ngoài checkout, phải từ chối đơn đã có dòng order_web3 (mã WEB3_ORDER_MANAGED) và kiểm quyền chủ đơn. Tồn kho: đơn Web3 trừ kho khi tạo (giữ chỗ) và hoàn kho khi EXPIRED hoặc REFUNDED. Quy tắc: chỉ ghi khi chuyển trạng thái hợp lệ; orders cũ không bao giờ được dùng để suy ra trạng thái đơn Web3; replay đặt lại cả hai bảng theo cùng bảng ánh xạ.

7.5. Ràng buộc toàn vẹn

- amount_raw không bao giờ chuyển sang FLOAT hay DOUBLE ; so sánh bằng số nguyên ở ứng dụng.
- UNIQUE(onchain_order_id), UNIQUE(address) ở user_wallets, UNIQUE(tx_hash, log_index) ở chain_events là lớp phòng thủ cuối cùng cho tính idempotent.
- Chuyển trạng thái dùng câu lệnh có điều kiện: UPDATE order_web3 SET status=:new WHERE order_id=:id AND status=:old; rowcount = 0 nghĩa là đã có người chuyển trước.
- Mọi truy vấn SQL tham số hóa, không nối chuỗi.

7.6. Migration và replay

Các tệp SQL đánh số (`001_wallets.sql` ...) chạy theo thứ tự, ghi `schema_migrations`, mỗi tệp idempotent; `001` kiểm tra kiểu `orders.id` trước khi tạo khóa ngoại. Dữ liệu mẫu `category_policies` chạy trong `bootstrap.sh`. Chế độ replay `--from-block 0 --reset` đặt lại các cột dẫn xuất của `order_web3` và trạng thái chiều ở `orders`, nhưng không đụng tới cột sinh từ báo giá (giá VND, `doc_hash`, nhóm hàng, `fee_bps`).

8. Thiết kế mật mã và EIP-712

8.1. Tập kiểu dùng chung

`shared/eip712/types.json` chứa domain và hai kiểu, theo đúng thứ tự trường của SRS mục 6:

Thành phần	Định nghĩa
Domain	<code>name = SlopeeEscrow</code> , <code>version = 1</code> , <code>chainId</code> (lấy từ <code>deployments/local.json</code>), <code>verifyingContract</code> (cùng tệp)
OrderQuote	<code>orderId</code> , <code>buyer</code> , <code>merchant</code> , <code>token</code> , <code>amount</code> , <code>inspectionDuration</code> , <code>disputeTimeout</code> , <code>feeBps</code> , <code>docHash</code> , <code>deadline</code> . Ký bởi <code>QUOTE_SIGNER_ROLE</code> . Struct <code>Quote</code> trong ABI có cùng danh sách và thứ tự trường.
DisputeResolution	<code>orderId</code> , <code>payoutTo</code> , <code>amount</code> , <code>nonce</code> , <code>deadline</code>

Python (`encode_typed_data`), ethers.js (`signer.signTypedData`) và test Foundry (`vm.sign`) cùng đọc tệp này. Frontend chỉ được gửi `chainId` và `verifyingContract` từ tệp triển khai, không nhập tay.

8.2. Xử lý chữ ký và địa chỉ

- Chữ ký 65 byte (`r`, `s`, `v`), `v` chuẩn hóa về 27/28. Backend từ chối độ dài khác 65 byte.
- Địa chỉ chuẩn hóa chữ thường trước khi lưu và so sánh; địa chỉ đầu vào được kiểm định dạng (`0x` + 40 hex) trước.
- Số uint256 trong JSON truyền dưới dạng chuỗi thập phân.
- Backend không bao giờ ghi chữ ký người dùng vào log.

8.3. Vector kiểm thử chéo ngôn ngữ

`shared/eip712/vectors.json` chứa các bộ dữ liệu cố định kèm digest và chữ ký mong đợi (khóa thử của Anvil). Bộ kiểm thử Python, JS và Solidity đều phải tạo ra cùng digest. Thêm một vector cho `docHash` chuẩn hóa (mục 5.3c) và một vector cho thông điệp liên kết ví. Đây là hạng mục [Q] tuần 1-2 của SRS mục 11, và phải xong trước khi đóng băng contract.

9. Thiết kế Frontend (React)

9.1. Công nghệ và tổ chức mã

Vite + React 19 (JavaScript, đúng stack hiện có của Slopee), React Router 7, ethers.js v6 là thư viện chạy duy nhất được thêm, và một hook `usePolling` tự viết cho trạng thái máy chủ (không thêm TanStack Query hay TypeScript). Trạng thái ví nằm trong một React context duy nhất; không lưu dữ liệu nhạy cảm vào `localStorage`.

```
frontend/src/
  lib/      client.js (fetch chung, gán Authorization, xử lý 401), chain.js
           (địa chỉ, ABI từ deployments), eip712.js (đọc types.json), errors.js,
           format.js (formatVND)
  hooks/    useWallet, useEscrow, useOrderStatus, useSignDispute,
           useServerClock, usePolling, useAuth
  pages/    wallet/, checkout/, orders/, disputes/, shipper/, merchant/,
  admin/
  components/ CountdownTimer, TxStepper, StatusBadge, AddressTag, AmountView
  mocks/    fetch mock theo hợp đồng API (tuần 1-3, không dùng MSW)
```

9.2. Màn hình và tuyến đường

Vai trò	Tuyến	Chức năng chính	Hook / API chính
Chung	<code>/wallet</code>	Kết nối và liên kết ví (UC-00)	<code>useWallet</code> , POST <code>/api/wallet/nonce</code> , <code>/link</code>

Vai trò	Tuyên	Chức năng chính	Hook / API chính
Buyer	<code>/checkout/:groupId</code>	Giá VND, số token, tỷ giá, hạn báo giá, hai bước approve tổng + nạp gom, hiển thị tổng cả giỏ và từng shop	useEscrow, GET <code>/api/orders/:code</code>
Buyer	<code>/orders/:code</code>	Trạng thái, đồng hồ kiểm tra, Đã nhận hàng, Khiếu nại, Hủy khi quá hạn	useOrderStatus
Buyer, Merchant	<code>/disputes/:code</code>	Xem đề xuất, ký EIP-712	useSignDispute
Merchant	<code>/merchant/orders</code> , <code>/merchant/pending</code>	Danh sách đơn Web3, rút khoản chờ rút	useEscrow.claimPending
Shipper	<code>/shipper/orders</code>	Đơn LOCKED, nút Xác nhận phát hàng	POST <code>/api/shipper/orders/:code/deliver</code>
Admin	<code>/admin/disputes</code> , <code>/admin/settings</code> , <code>/admin/reconcile</code>	Xử lý khiếu nại, cấu hình phí và chính sách, đối soát, cảnh báo	Các API admin

9.3. Máy trạng thái nạp tiền (Buyer)

Bước	Điều kiện sang bước kế	Lỗi thường gặp và cách xử lý
CHECK_NETWORK	<code>chainId</code> ví bằng <code>chainId</code> triển khai	Yêu cầu <code>wallet_switchEthereumChain</code>
CHECK_QUOTE	Còn ít nhất 60 giây tới <code>deadline</code> chung của lô	Hết hạn hoặc <code>FEE_CAP_CHANGED</code> : gọi <code>POST /api/checkout-groups/:id/quote</code> lấy báo giá mới cho cả lô
CHECK_ALLOWANCE	<code>allowance >= total_amount_raw</code> thì bỏ qua APPROVING	Số dư MockUSD không đủ cho tổng: báo rõ số thiếu, không gửi
APPROVING	Biên nhận thành công (approve tổng)	Người dùng từ chối (mã 4001): quay về, không báo lỗi hệ thống
DEPOSITING	Biên nhận thành công (<code>depositEscrowBatch</code>)	Revert: ánh xạ lý do sang thông báo (9.5); cả lô thất bại, không đơn nào được tạo
WAITING_INDEXER	Polling <code>GET /api/checkout-groups/:id</code> thấy mọi đơn con LOCKED	Quá 30 giây: hiện mã giao dịch và gợi ý kiểm tra Indexer

Giao diện **không** đặt trạng thái LOCKED theo biên nhận của ví; chỉ tiến tới LOCKED khi API trả về (DD-04, BR-09).

Sau khi lô LOCKED, giao diện hiển thị các đơn con theo shop (mỗi shop một thẻ với trạng thái riêng). Mỗi đơn có đồng hồ kiểm tra, nút Đã nhận hàng và nút Khiếu nại riêng; thao tác trên đơn của shop này không đổi trạng thái đơn của shop khác. Giả một shop dùng cùng luồng với một đơn con.

9.4. Đồng hồ và thời gian

Trả lời của `GET /api/orders/:code/status` kèm `server_time`. Frontend tính độ lệch đồng hồ một lần (`useServerClock`) rồi đếm ngược tới `delivered_at + inspection_duration`, tránh sai lệch múi giờ hay đồng hồ máy khách. Khi đếm về 0, nút Khiếu nại bị vô hiệu hóa nhưng vẫn để contract là bên quyết định cuối.

`server_time` lấy từ `now_chain()` (R-09), cùng đồng hồ với hạn báo giá, nên bước CHECK_QUOTE vẫn đúng khi Anvil bị tua thời gian.

9.5. Ảnh xạ lỗi

Nguồn	Nhận diện	Thông báo cho người dùng
MetaMask	mã 4001	Bạn đã hủy thao tác.
Contract	<code>Quote expired</code>	Báo giá đã hết hạn, hãy lấy báo giá mới.
Contract	<code>Order exists</code>	Đơn này đã được thanh toán.
Contract	<code>Under inspection</code> , <code>Expired</code>	Chưa hết hoặc đã quá thời hạn kiểm tra.
Contract	<code>Need two distinct parties</code>	Chữ ký chưa hợp lệ, hãy tải lại đề xuất.
API	<code>code</code> trong thân trả lời	Thông báo tiếng Việt từ backend.

Ảnh xạ bổ sung: Invalid batch size hoặc mã BATCH_TOO_LARGE hiện là Mỗi lần thanh toán tối đa 5 shop, hãy bỏ bớt sản phẩm; `Fee exceeds on-chain cap` hoặc mã `FEE_CAP_CHANGED` hiện là Phí sàn đã thay đổi, hãy lấy báo giá mới; `Caller must be quote buyer` hiện là Báo giá này không thuộc ví đang kết nối. Lỗi `Deadline must be before dispute timeout` chỉ ghi log vì backend luôn ký hạn hợp lệ.

9.6. Làm việc song song với backend

Tuần 1-3 frontend dùng fetch mock giả lập API theo hợp đồng mục 10; fixture đủ cho 7 trạng thái đơn và các mã lỗi. Từ tuần 4 tắt mock theo từng endpoint khi backend sẵn sàng.

9.7. Thông báo khi ký phán quyết

Trên `/disputes/:code`, Buyer và Merchant luôn thấy khung cảnh báo màu vàng ngay trước nút ký (R-06), cùng kết quả sẽ xảy ra (hoàn tiền hay giải ngân), số tiền và địa chỉ nhận:

Lưu ý: Sàn đã ký phê duyệt đề xuất này. Sau khi bạn ký xác nhận, bất kỳ ai (kể cả relayer tự động của sàn) cũng có thể nộp chữ ký lên blockchain để chuyển tiền ngay lập tức.

9.8. Điều chỉnh theo frontend hiện có của Slopee

Frontend thật là React 19, JavaScript, React Router 7, `fetch` trực tiếp với địa chỉ `http://localhost:5000` lập ở từng tệp `api/*.js`, và phiên đăng nhập là một object người dùng trong `localStorage`. Thiết kế giữ nguyên stack này; nếu muốn kiểm kiểu thì dùng chú thích JSDoc thay vì chuyển sang TypeScript.

Tệp hiện có	Thay đổi	Gốc
api/auth.js , payments.js , carts.js , shops.js , products.js , admin.js	Bỏ hardcoded địa chỉ (đọc VITE_API_BASE), bỏ tham số userId , gọi qua lib/client.js	P0-01
pages/Login.jsx , Signup.jsx	Nhận token lưu ở sessionStorage ; Signup chỉ cho chọn user hoặc seller	P0-01, P0-02
App.jsx	Bọc các route bằng RequireRole ; thêm /wallet , /checkout/group/:id , /shipper , /disputes/:code	P0-01
components/Navbar.jsx	Nút kết nối ví, địa chỉ rút gọn, giá VND	UC-00
Home.jsx , ProductView.jsx , Cart.jsx	Dùng formatVND , bỏ \$ và toFixed(2)	P0-07
pages/Checkout.jsx	Thêm phương thức Web3: từng shop và tổng, TxStepper (approve tổng, nạp gom, chờ Indexer), thông báo BATCH_TOO_LARGE	UC-01, UC-02
pages/MyOrders.jsx	Nhóm theo checkout_group_id , huy hiệu Khiếu nại Web3, đồng hồ kiểm tra, nút Đã nhận hàng và Khiếu nại, khóa nút cũ với đơn Web3	BR-11, Q-07
pages/SellerDashboard.jsx	Ô giá nhận số nguyên VND, liên kết ví nhận tiền, danh sách đơn Web3, rút khoản chờ rút	P0-07, UC-00
pages/AdminDashboard.jsx	Khiếu nại, đề xuất phán quyết, phí, ánh xạ danh mục, đối soát, cảnh báo	UC-06, UC-09

Tệp mới: pages/Wallet.jsx , pages/ShipperOrders.jsx , pages/Dispute.jsx , components/RequireRole.jsx , components/TxStepper.jsx , lib/client.js , lib/format.js , lib/chain.js , lib/eip712.js , lib/errors.js , hooks/* (mục 9.1).

Hợp đồng của các hàm tách riêng để dễ kiểm thử. `depositReducer(state, event)` là hàm thuần cài đặt máy trạng thái ở 9.3; `Checkout.jsx` chỉ gọi hàm này và thực hiện hiệu ứng (ví, API). `usePolling(fn, {intervalMs, stopWhen})` dừng khi rời trang, không chồng lệnh gọi và dừng khi đạt trạng thái cuối. `client.js` gắn `Authorization`, và khi gặp 401 thì xóa token rồi chuyển về đăng nhập. `formatVND` là nơi duy nhất định dạng tiền.

10. Thiết kế API và luồng tuần tự

10.1. Quy ước chung

Xác thực bằng token phiên do lớp xác thực mới cấp (Slopee hiện chưa có phiên, xem mục 16); mọi định danh người dùng lấy từ token, không nhận từ thân yêu cầu hay đường dẫn. Mọi trả lời lỗi có dạng `{code, message}`. Số tiền trả về dạng chuỗi (`amount_raw`) kèm `decimals` và `symbol`. Thời gian trả về dạng giây UTC (số nguyên). Mọi endpoint ghi dữ liệu kiểm tra vai trò và quyền sở hữu đơn ở lớp ứng dụng, không chỉ ở giao diện.

10.2. Hợp đồng các endpoint then chốt

Endpoint	Đầu vào	Đầu ra	Ghi chú
POST <code>/api/wallet/nonce</code>	không	<code>nonce</code> , <code>message</code> , <code>expires_at</code>	Giới hạn tần suất theo tài khoản.
POST <code>/api/wallet/link</code>	<code>address</code> , <code>signature</code>	<code>address</code> , <code>linked_at</code>	Lỗi: <code>NONCE_INVALID</code> , <code>BAD_SIGNATURE</code> , <code>WALLET_TAKEN</code> .
POST <code>/api/orders</code>	<code>cart_item_ids</code> (nhiều shop)	<code>checkout_group_id</code> , <code>total_amount_raw</code> , <code>decimals</code> , <code>symbol</code> , <code>deadline</code> , mảng <code>orders</code> gồm <code>order_id</code> , <code>shop_id</code> , <code>quote</code> , <code>signature</code> , <code>amount_raw</code> , <code>price_vnd</code> , <code>fx</code> , <code>inspection_duration</code> , <code>dispute_timeout</code> , <code>fee_bps</code>	Nhóm theo shop; tối đa 5 shop (vượt thì 422 <code>BATCH_TOO_LARGE</code>); tổng mỗi shop là số nguyên VND.

Endpoint	Đầu vào	Đầu ra	Ghi chú
POST /api/checkout-groups/:id/quote	không	như POST /api/orders	Cập lại cho cả lô khi hết hạn hoặc <code>FEE_CAP_CHANGED</code> .
GET /api/checkout-groups/:id	không	danh sách đơn con và trạng thái từng đơn	Chỉ chủ lô.
GET /api/orders/:code/status	không	<code>status</code> , <code>delivered_at</code> , <code>inspection_duration</code> , <code>disputed_at</code> , <code>dispute_timeout</code> , <code>anomaly</code> , <code>server_time</code>	Nhẹ, dùng cho polling 2-3 giây.
POST /api/disputes/:code/signatures	<code>resolution_id</code> , <code>signature</code>	<code>status</code> của đề xuất (PROPOSED hoặc READY)	Bên được xác định từ token đăng nhập, không do client chọn.
POST /api/disputes/:code/submit	không	<code>tx_hash</code>	Chỉ khi READY và chưa hết hạn.
PUT /api/admin/category-policies/:code	<code>inspection_duration</code> , <code>dispute_timeout</code>	chính sách đã lưu	Kiểm khoảng 1 giờ-30 ngày và 7-90 ngày, chỉ ảnh hưởng đơn mới.
PUT /api/admin/category-policy-map/:categoryId	<code>policy_code</code>	ánh xạ đã lưu	Chỉ ảnh hưởng đơn mới.

10.3. SEQ-01: Ký quỹ

```

Buyer(UI) -> API: POST /api/orders (giỏ nhiều shop)
API -> DB: checkout_groups + mỗi shop: orders(pending), order_web3,
order_quotes
API -> Signer: sign_quote cho từng shop (cùng deadline)
API -> Buyer(UI): checkout_group_id, total_amount_raw, orders[] (quote +
signature)
Buyer(UI) -> MockUSD: approve(escrow, total_amount_raw) [MetaMask 1]
Buyer(UI) -> Escrow: depositEscrowBatch(quotes, sigs) [MetaMask 2]
Escrow: kiểm từng báo giá, ghi từng đơn LOCKED, thu tổng tiền một lần, emit N
lần OrderCreated
        (nguyên tử: một báo giá lỗi thì cả lô revert)
Indexer -> Escrow: get_logs; mỗi OrderCreated: order_web3 = LOCKED, orders =
paid, xóa cartItems của đơn
Buyer(UI) -> API: polling GET /api/checkout-groups/:id -> các đơn con LOCKED

```

10.4. SEQ-02: Tranh chấp và giải quyết

```

Buyer -> Escrow: raiseDispute ; Indexer: DISPUTED
Buyer(UI) -> API: POST dispute-note (lý do, bằng chứng)
Admin -> API: POST proposal(payoutTo) ; API: đọc nonce, tính deadline,
Signer ký Arbitrator, PROPOSED
Buyer/Merchant -> MetaMask: signTypedData_v4 ; -> API: POST signatures ; API:
xác minh, READY
Relayer/bên được lợi -> Escrow: resolveDispute(payoutTo, deadline, sig1,
sig2)
Escrow: _settle, emit OrderRefunded|OrderCompleted, DisputeResolved
Indexer: EXECUTED

```

10.5. SEQ-03: Hết hạn kiểm tra tự giải ngân

```

AutoRelease(job) -> Chain: so sánh thời gian block mới nhất với deliveredAt +
inspectionDuration
AutoRelease -> Escrow: releaseAfterInspection(orderId) [relayer trả gas]
Escrow: COMPLETED, chia phí theo feeBps
Indexer: DB COMPLETED

```

11. Thiết kế bảo mật

11.1. Mỗi đe dọa và biện pháp

Mỗi đe dọa (BRD mục 6)	Biện pháp trong thiết kế	Kiểm chứng
Chiếm mã đơn, đổi Merchant, sai số tiền	Báo giá EIP-712 ràng buộc người nạp; mã đơn dùng một lần	T-01 đến T-04
Dùng lại chữ ký phán quyết	<code>nonce</code> theo đơn, <code>deadline</code> , ràng buộc <code>amount</code> và <code>payoutTo</code>	T-12, T-13
Một bên nắm hai phiếu	Tính phiếu theo bên; cầm Trọng tài làm Buyer/Merchant	T-04, T-12
Reentrancy qua ví nhận	CEI, <code>nonReentrant</code> , giới hạn gas 30.000, pull-payment	T-16, T-17
Lộ khóa vận hành sàn	Bốn khóa tách vai trò, nạp từ môi trường, kiểm khởi động, che nhật ký	E-05, Slither
Shipper khai gian	Thông báo Buyer ngay khi giao, cửa sổ kiểm tối thiểu 1 giờ	Giả định tin cậy (BRD 6)
Giả mạo trạng thái ở giao diện	Trạng thái chỉ do Indexer ghi; API không có endpoint ghi trạng thái từ client	I-03
Lạm dụng API (<code>nonce</code> , báo giá)	Giới hạn tần suất, phiên đăng nhập, kiểm quyền sở hữu đơn	Test API

11.2. Quản lý khóa

Bốn khóa (báo giá, Trọng tài, Shipper và khóa gửi giao dịch nếu tách) lưu ở biến môi trường hoặc tệp bí mật ngoài kho mã (`.env` nằm trong `.gitignore`, kiểm bằng pre-commit hook quét chuỗi 64 hex). Không đặt khóa Admin trong máy chủ; Admin ký giao dịch quản trị bằng ví của họ. Phase 2: KMS/HSM, multisig, giới hạn tần suất ký.

11.3. Danh sách kiểm

- **Contract:** CEI ở mọi đường chuyển tiền; mọi hàm công khai có kiểm quyền và trạng thái; không có vòng lặp vô hạn; số học 0.8.x có kiểm tràn; phân tích Slither không còn cảnh báo cao chưa giải thích.

- **Backend:** truy vấn tham số hóa; kiểm dữ liệu đầu vào (địa chỉ, số tiền, độ dài chữ ký); giao dịch CSDL có điều kiện trạng thái; không ghi chữ ký hay khóa vào nhật ký; phiên bản thư viện được khóa.
- **Frontend:** không dùng `eval`, không chèn HTML thô từ dữ liệu người dùng (lý do và đường dẫn bằng chứng phải được thoát ký tự); luôn hiển thị địa chỉ nhận và số tiền trước khi ký.

12. Triển khai và vận hành

12.1. Dựng môi trường một lệnh

`./scripts/bootstrap.sh` thực hiện theo thứ tự: kiểm công cụ (Foundry, Python, Node, MySQL); khởi động Anvil với `--block-time 2` (block đều đặn để đồng hồ chuỗi không bị kẹt); `forge build` và triển khai (mục 4.7); ghi `deployments/local.json`; chạy migration và dữ liệu mẫu; sinh `.env` mẫu từ tài khoản Anvil (bốn khóa vai trò khác nhau); mint MockUSD; khởi động web, indexer, scheduler và frontend.

12.2. Biến môi trường

Biến	Ý nghĩa
<code>RPC_URL</code> , <code>CHAIN_ID</code>	Kết nối Anvil
<code>ESCROW_ADDRESS</code> , <code>MOCKUSD_ADDRESS</code>	Đọc từ <code>deployments/local.json</code>
<code>QUOTE_SIGNER_KEY</code> , <code>ARBITRATOR_KEY</code> , <code>SHIPPER_KEY</code>	Khóa vận hành (mục 2.2 SRS)
<code>DB_URL</code>	Kết nối MySQL
<code>POLL_SECONDS</code> , <code>CONFIRMATIONS</code> , <code>REORG_DEPTH</code>	Tham số Indexer

Tỷ giá, hạn báo giá, hạn chữ ký nằm ở `platform_settings` chứ không ở biến môi trường, để Admin đổi không cần khởi động lại.

12.3. Kiểm tra sức khỏe và sổ tay xử lý sự cố

Triệu chứng	Nguyên nhân thường gặp	Hành động
Đơn không lên LOCKED sau 10 giây	Indexer dừng hoặc mất khóa <code>GET_LOCK</code>	Xem nhật ký Indexer; khởi động lại; kiểm <code>indexer_cursor</code>
Báo giá bị contract từ chối chữ ký	Lệch chainId hoặc địa chỉ contract giữa các tầng	Chạy test chéo ngôn ngữ; đọc lại <code>deployments/local.json</code>
Giao dịch relayer lỗi nonce	Hai tiến trình dùng cùng khóa	Chỉ một relayer; kiểm <code>tx_outbox</code>
Báo giá luôn hiện hết hạn sau khi tua thời gian	Đồng hồ hệ thống chậm hơn thời gian chuỗi (R-09)	Dùng <code>now_chain</code> cho báo giá, chạy Anvil với <code>--block-time 2</code> , cấp báo giá mới sau khi tua
Indexer báo Anvil khởi động lại	Mất trạng thái chuỗi	Chạy <code>bootstrap.sh</code> rồi replay

13. Thiết kế kiểm thử

13.1. Các tầng kiểm thử

Tầng	Công cụ	Phạm vi	Mốc
Contract: đơn vị, fuzz, invariant	Foundry (<code>forge test</code> , <code>vm.warp</code> , <code>vm.sign</code>)	T-01 đến T-34, INV-01 đến INV-07	M1
Miền backend	pytest, ChainPort giả trong bộ nhớ, ClockPort giả	Quy đổi, chọn chính sách, máy trạng thái, docHash	M2
Backend trên MySQL thật	pytest với dịch vụ MySQL của CI, fixture người dùng và token	A-01 đến A-07, I-06; mọi đường tiền, tồn kho, phân quyền	Giai đoạn 0, M3
Adapter và Indexer	pytest với Anvil thật hoặc nhật ký ghi sẵn	I-01 đến I-07, replay, trùng sự kiện	M2-M3
API	pytest + client Flask	Quyền, mã lỗi, ca âm	M3
Chéo ngôn ngữ	pytest, Vitest và Foundry cùng đọc <code>vectors.json</code>	Digest EIP-712 và docHash khớp ở 3 nơi	Tuần 2

Tầng	Công cụ	Phạm vi	Mốc
Frontend đơn vị	Vitest + Testing Library	F-01 đến F-06 và các test hiện có	Từ tuần 1
End-to-end	Playwright (Firefox) với ví thử nghiệm	E-01 đến E-07	M4
Kiến trúc	pytest kiểm import	Miền không phụ thuộc hạ tầng (T-ARCH-01)	M2

Bản demo thật dùng MetaMask; E2E tự động dùng một nhà cung cấp ví (EIP-1193) chạy bằng khóa thử của Anvil và chỉ nạp trong môi trường kiểm thử, nhờ đó chạy được trong CI mà không phụ thuộc tiện ích trình duyệt.

13.2. Thiết kế bộ test contract

- Một tập `Base.t.sol` dựng sẵn bảy tài khoản (Buyer, Merchant, Shipper, Trọng tài, người ký báo giá, Admin, nhận phí), MockUSD đã mint, và hàm tiện ích `signQuote`, `signDispute`, `deposit`, `warpPastInspection`.
- Mỗi cặp (trạng thái, hàm) của bảng SRS mục 5 có một ca âm (T-19), sinh bằng vòng lặp qua danh sách hàm.
- Handler cho invariant thực hiện chuỗi ngẫu nhiên các hàm hợp lệ (nạp, giao, giải ngân, khiếu nại, giải quyết, hủy) để kiểm INV-01 đến INV-06.
- Ca biên thời gian: đúng `deliveredAt + inspectionDuration` và trước đó 1 giây (T-07, T-10); đúng 7 và 90 ngày (T-20).

13.3. Cổng chất lượng

Cổng	Ngưỡng
Độ phủ dòng lệnh contract	Từ 90% (<code>forge coverage</code>)
Slither	Không còn cảnh báo cao chưa giải thích
Test chéo ngôn ngữ	100% vector khớp
Demo lặp lại	3 lần liên tiếp không lỗi (AC-11)
Mỗi PR	Một thành viên khác xem lại, toàn bộ test xanh

13.4. Ca kiểm thử bổ sung sau rà soát

- **T-22 (phí):** báo giá có `feeBps` lớn hơn `defaultFeeBps` thì revert; bằng đúng trần thì thành công; sau khi Admin hạ trần, báo giá cũ phí cao bị revert, báo giá mới thành công; phí trong sự kiện `OrderCreated` bằng phí trong báo giá.
- **T-23 (người nạp):** `q.buyer` khác `msg.sender` thì revert; sửa `buyer` hoặc `feeBps` sau khi ký thì lỗi `Bad quote signature`.
- **T-24 (hạn chữ ký):** `resolveDispute` với `deadline` bằng hoặc lớn hơn `disputedAt + disputeTimeout` thì revert; bằng mốc trừ 1 giây thì thành công.
- **T-15 mở rộng:** tại thời gian mốc trừ 1, `arbitratorForceResolve` revert còn `resolveDispute` (`deadline` = mốc trừ 1) thành công; tại đúng mốc, `arbitratorForceResolve` thành công còn `resolveDispute` revert vì chữ ký hết hạn.
- **INV-07:** thêm vào handler invariant, kiểm phí của mọi đơn không vượt trần tại lúc nạp.
- **I-05 (đồng hồ):** sau `evm_increaseTime`, báo giá mới ký bằng `now_chain` được contract chấp nhận; Anvil chạy `--block-time 2`; `server_time` khớp `now_chain`.
- **I-06 (đồng bộ ngược):** mỗi chuyển trạng thái Web3 cập nhật đúng `orders` cũ trong cùng giao dịch; replay đặt lại cả hai bảng và khớp 100% với chuỗi.
- **Vector chéo ngôn ngữ:** bộ `vectors.json` gồm cả trường `buyer` và `feeBps` của `OrderQuote`.

Ca kiểm thử nạp gom (v1.2).

- **T-25:** nạp gom 3 đơn của 3 Merchant: đúng một lần `transferFrom`, số dư tăng đúng tổng, 3 `OrderCreated`, mỗi đơn LOCKED với phí và thời hạn riêng.
- **T-26, T-27:** lô rỗng, quá 5 đơn, lệch độ dài mảng; hai báo giá cùng `orderId` thì revert và số dư không đổi.
- **T-28:** một báo giá hết hạn, sai chữ ký, phí vượt trần hoặc `buyer` khác thì cả lô revert, không đơn nào được tạo.
- **T-29, T-30:** nhiều loại token, sai tổng ETH; mọi kiểm tra của `depositEscrow` áp dụng cho từng phần tử lô (số tiền 0, Merchant trùng Buyer, Trọng tài làm Merchant, thời hạn ngoài khoảng).
- **T-31:** độc lập: lô 3 đơn, đơn A mở khóa sớm, đơn B khiếu nại rồi hoàn tiền, đơn C hết hạn tự giải ngân; mỗi đơn đúng số tiền và phí.
- **T-32:** lô đúng 5 đơn thành công; ghi lại gas (mục tiêu dưới 3 triệu).
- **T-33, T-34:** fuzz và invariant có `depositEscrowBatch`; `depositEscrow` đơn lẻ và lô một phần tử cho kết quả giống hệt.

- **I-07, E-07:** giỏ 3 shop ở backend (tổng bằng tổng số token làm tròn lên từng shop, cập lại báo giá cả lô, hết hạn cả lô) và luồng end-to-end chỉ cần 2 lần xác nhận ví.

13.5. Chuyển đổi bộ kiểm thử sẵn có và CI

Repo Slopee hiện có pytest với mock con trỏ CSDL (5 tệp, khoảng 340 dòng), Vitest cho Home, Navbar và logic Tetris, Playwright chỉ trên Firefox (đăng ký và Tetris, dọn dữ liệu bằng kết nối MySQL trực tiếp), và CI GitHub Actions đã có dịch vụ MySQL.

Test hiện có	Thay đổi bắt buộc	Nguyên nhân
<code>test_checkout.py</code>	Viết lại: gửi token thay <code>userId</code> , giá VND số nguyên; các ca race tồn kho và trừ số dư chạy trên MySQL thật vì mock không kiểm chứng được	P0-01, 05, 07
<code>test_cart.py</code> , <code>test_products.py</code> , integration	Thêm token và ca 401, 403; bỏ <code>userId</code> khỏi thân	P0-01, 03
<code>auth_test.py</code>	Đăng ký chỉ nhận <code>user</code> và <code>seller</code> ; đăng nhập trả token; giới hạn số lần thử	P0-01, 02, 05
<code>Home.test.jsx</code> , <code>Navbar.test.jsx</code>	Giá theo VND; Navbar có nút ví; mock <code>fetch</code> qua client chung	P0-07, UC-00
<code>auth-flow.spec.js</code>	Theo luồng token; dọn dữ liệu bằng hàm tiện ích dùng chung	P0-01
<code>tetris-flow.spec.js</code> , <code>tetrisLogic.test.js</code>	Giữ nguyên, chỉ thêm đăng nhập bằng token nếu route yêu cầu	-

Nguyên tắc. Mock con trỏ CSDL chỉ dùng cho logic thuần. Mọi đường liên quan tiền, tồn kho, trạng thái đơn và phân quyền chạy trên MySQL thật, kèm một fixture tạo người dùng và token (đây là lý do ca A-05, race tồn kho, không làm được bằng mock).

Ví thử nghiệm cho E2E. MetaMask thật không tự động hóa được trong CI, nên Playwright thêm một nhà cung cấp ví `window.ethereum` (EIP-1193) chạy bằng khóa thử của Anvil qua `addInitScript`, chỉ nạp trong môi trường kiểm thử. Bản demo thủ công vẫn dùng MetaMask.

Công việc CI	Nội dung
backend	Có sẵn: pytest với dịch vụ MySQL; thêm các test A-xx trên MySQL thật
frontend	Thêm: <code>npm run lint</code> , <code>npm test</code> (Vitest, gồm F-01 đến F-06)
contracts	Mới: <code>forge test</code> , <code>forge coverage</code> (từ 90%), Slither
web3-integration	Mới: Anvil, triển khai, Indexer, backend, kiểm thử I-xx và chéo ngôn ngữ
e2e	Mới: Anvil, backend, frontend, Playwright với ví thử nghiệm (E-01 đến E-07)

14. Truy vết yêu cầu tới thiết kế

BRD	SRS (use case, kiểm thử)	Phần SDD
BR-01	UC-00, E-05	2.1, 5.2, 5.6, 11.2
BR-02	UC-04, UC-09, T-08, T-09	4.3, 4.6 (INV-03, 06), R-01
BR-03	UC-03 đến UC-05, T-05 đến T-07, T-10	4.3, 5.3h, 9.4
BR-04	UC-06, T-11 đến T-13	4.5, 5.3i, 8
BR-05	UC-07, T-14 đến T-16, T-20, T-21	4.4, 5.5, R-02, R-09
BR-06	UC-01, UC-02, T-01 đến T-04	5.3e, 5.3f, 8.1
BR-07	UC-00, I-01	5.3g
BR-08	UC-01, I-02	5.3a
BR-09	UC-08, I-03, E-06	6, 7.1
BR-10	UC-01, T-20, T-21, I-04	5.3b, 6.2, 7.2

Bổ sung BR-11 (giỏ nhiều shop, nạp gom): UC-01, UC-02; thiết kế ở mục 4.9, 5.3e, 7.2, 9.3, 10.2, 10.3; kiểm thử T-25 đến T-34, I-07, E-07; nghiệm thu AC-14.

15. Phát hiện khi rà soát và quyết định đã chốt

Nhóm đã rà soát và chốt các phát hiện R-01 đến R-09 cùng quyết định Q-01 đến Q-05 như dưới đây. Cột Áp dụng chỉ các mục SDD đã được cập nhật.

15.1. Kết luận đã chốt

Mã	Kết luận	Áp dụng
R-01, Q-03	Thêm <code>buyer</code> và <code>feeBps</code> vào <code>Quote</code> và <code>OrderQuote</code> . <code>defaultFeeBps</code> giữ làm trần phí on-chain. Backend ký <code>feeBps = min(PLATFORM_FEE_BPS, trần)</code> ; báo giá vượt trần bị từ chối (<code>FEE_CAP_CHANGED</code>).	4.3, 4.5, 4.8, 5.3e, 5.4, 6.2, 8.1
R-02, Q-05	Contract ép <code>deadline < disputedAt + disputeTimeout</code> ; backend ký <code>deadline = min(now_chain + TTL, disputedAt + disputeTimeout - 1)</code> ; có ca biên mốc trừ 1 và đúng mốc.	4.3, 4.8, 5.3i, 13.4
R-03	ID bắt đầu từ 100001, <code>CHECK</code> nhỏ hơn 2^{53} ; frontend dùng chuỗi hoặc <code>BigInt</code> .	7.2, 7.3
R-04	Thêm <code>chain_events.block_timestamp</code> ; đếm theo block; mọi mốc lấy từ thời gian block.	6.1, 7.2, 7.3
R-05	Struct <code>Quote</code> và kiểu <code>OrderQuote</code> giống hệt nhau (có <code>buyer</code>); vector chéo ngôn ngữ.	4.5, 8.1, 13.4
R-06	Giữ thiết kế; hiện thông báo vàng trước khi ký.	9.7
R-07, Q-04	Bảng <code>order_web3</code> quan hệ 1-1; đồng bộ ngược sang <code>orders</code> cũ trong cùng giao dịch.	6.2, 7
R-08	Điều chỉnh sau khi đọc mã: Slopee không có vận đơn nên thêm vai trò shipper và bảng <code>shipper_assignments</code> .	7.3
R-09	<code>now_system()</code> và <code>now_chain()</code> ; Anvil chạy <code>--block-time 2</code> .	5.2, 5.3, 12
Q-01	MockUSD dùng 6 chữ số thập phân.	4.1

Mã	Kết luận	Áp dụng
Q-02	Giữ 4 nhóm hàng theo BR-10 của BRD; đây vẫn là giá trị đề xuất minh họa, báo cáo cuối kỳ nêu cơ sở chọn.	5.3b, <code>category_policies</code>

15.2. Vấn đề còn mở

Mã	Nội dung	Trạng thái
Q-06	Kiểu <code>orders.id</code> và migration	Đã chốt: <code>VARCHAR(36)</code> , migration dò kiểu, charset, collation, engine từ <code>information_schema</code> , khóa ngoại <code>ON DELETE RESTRICT</code> (7.2).
Q-07	Hiển thị DISPUTED ở giao diện cũ	Đã chốt: giữ <code>shipped</code> , huy hiệu vàng từ <code>order_web3</code> , chặn đường ghi cũ (7.4).
Q-08	Giỏ hàng nhiều shop	Đã chốt: không chặn; thanh toán gom một lần, tách đơn độc lập theo shop (4.9, BR-11).
Q-09	Đơn vị tiền của Slopee	Đã chốt: VND số nguyên trên toàn nền tảng, không còn USD (16.3, bước 8).
Q-10	Cơ chế xác thực	Đã chốt: token ký có hạn gửi qua header <code>Authorization</code> (16.3, bước 1).
Q-11	Giới hạn số shop mỗi lần nạp gom	Đã chốt: tối đa 5 shop vì đây là bản demo; vượt thì yêu cầu bỏ bớt sản phẩm (<code>BATCH_TOO_LARGE</code>). Có thể nâng lên 10 sau khi đo gas ở T-32; tách tự động thành nhiều lô là Phase 2.
Q-12	Thời điểm xóa sản phẩm khỏi giỏ	Đã chốt: chỉ xóa khi đơn LOCKED, tức tiền đã thật sự vào hợp đồng (6.2). Nếu người dùng hủy ở MetaMask thì giỏ còn nguyên.

15.3. Tài liệu liên quan cần đồng bộ

- **BRD:** đã nâng lên v5.2 (BR-02, BR-04, BR-06, mục 6, AC-04, AC-06, AC-13).
- **SRS lên v5.2:** UC-01 (báo giá có `buyer` và `feeBps`, đọc trên on-chain), UC-06 (công thức `deadline`), UC-09 (quy trình đổi phí), mục 6 (OrderQuote), mục 9 (`order_web3`, thời gian dạng `BIGINT`, `block_timestamp`), mục 10 (mã theo 4.8), mục 12 (sửa T-08, T-

09, T-15, T-21; thêm T-22 đến T-24, I-05, I-06), mục 14 (ma trận truy vết).

16. Hiện trạng mã nguồn Slopee và việc phải làm trước khi tích hợp

Mục này dựa trên việc đọc `backend/` của repo Slopee-main (schema, `app.py`, `init_db.py` và bảy tệp routes). Hai lỗi S-08 và S-13 đã được kiểm chứng bằng chạy thử; các lỗi còn lại dựa trên việc đọc mã.

16.1. Giả định của SDD so với thực tế

Chủ đề	SDD giả định	Thực tế trong Slopee	Xử lý
Xác thực	Có phiên đăng nhập	Không có phiên hay token; đăng nhập chỉ trả JSON người dùng, frontend lưu ở <code>localStorage</code> , backend tin <code>userId</code> gửi lên	Xây lớp xác thực (16.3)
Khóa chính	<code>INT</code>	<code>VARCHAR(36) UUID (users , orders , shops)</code>	Đổi kiểu (7.2, 7.3)
Trạng thái đơn	Viết hoa	<code>pending, paid, shipped, cancelled, received</code>	Bảng ánh xạ (7.4)
Shipper	Có vai trò và bảng vận đơn	<code>users.role</code> chỉ có <code>admin, seller, user</code> ; không có vận đơn	Thêm vai trò và bảng (7.3)
Nhóm hàng	Có mã nhóm	<code>categories(id, name)</code>	Bảng ánh xạ (7.3)
Merchant của đơn	Mỗi đơn một Merchant	<code>orders</code> không có merchant; một đơn gồm nhiều shop	Nạp gom, tách đơn theo shop (4.9)
Tiền	VND số nguyên	<code>DECIMAL(11, 2)</code> , tính bằng <code>float</code> ; giao diện hiển thị ký hiệu \$ (USD); số dư giả lập, tiền bán hàng cộng ngay cho người bán	Chuyển toàn bộ sang VND số nguyên (16.3, bước 8)
Migration	Tệp SQL đánh số	<code>init_db.py</code> tách <code>schema.sql</code> bằng dấu chấm phẩy; có route <code>HTTP /api/shops/migrate</code> chạy <code>ALTER</code>	Bộ chạy migration riêng

16.2. Lỗi bảo mật phát hiện

Mã	Mức	Vị trí	Vấn đề
S-01	Nghiêm trọng	Toàn bộ routes	Không endpoint nào xác thực. <code>userId</code> lấy từ thân yêu cầu hoặc đường dẫn nên ai cũng đọc được phương thức thanh toán (số tài khoản, số dư), đơn hàng, giỏ hàng của người khác.
S-02	Nghiêm trọng	<code>auth.py</code> đăng ký	<code>role</code> lấy thẳng từ client và ENUM cho phép <code>admin</code> , nên ai cũng tạo được tài khoản admin.
S-03	Nghiêm trọng	<code>admin.py</code>	Không kiểm quyền: liệt kê người dùng (email, điện thoại), xóa người dùng, sửa danh mục.
S-04	Nghiêm trọng	<code>payments.py</code> thêm phương thức	Mỗi lần gọi cấp 10.000 số dư giả lập cho <code>userId</code> tùy ý, nên tạo tiền vô hạn.
S-05	Cao	<code>payments.py</code> cập nhật đơn	Không kiểm chủ đơn, không kiểm trạng thái hiện tại. Hủy đơn không hoàn tiền hay tồn kho. Đây là đường ghi <code>orders.status</code> mà Web3 phải chặn.
S-06	Cao	<code>shops.py</code>	Sửa giá, tồn kho, xóa sản phẩm, thêm ảnh và biên thể không kiểm chủ shop; thêm sản phẩm vào shop bất kỳ.
S-07	Cao	<code>shops.py</code> /migrate	Route GET không xác thực chạy <code>ALTER TABLE</code> , hardcoded tên schema.
S-08	Cao	<code>payments.py</code> thanh toán	Kiểm kho rồi trừ kho không khóa và không điều kiện (race, bán lỗ); tiền tính bằng <code>float</code> ; trừ số dư không điều kiện; không có <code>SELECT ... FOR UPDATE</code> . Tài khoản không có PIN (như admin) gây lỗi 500: <code>check_password_hash(None, ...)</code> ném <code>AttributeError</code> (đã chạy thử) và API trả thông báo lỗi nội bộ cho client.
S-09	Trung bình	Đăng nhập, PIN 6 số	Không giới hạn số lần thử, nên dò mật khẩu và PIN được.

Mã	Mức	Vị trí	Vấn đề
S-10	Trung bình	app.py , config.py , init_db.py	debug=True , SECRET_KEY mặc định, admin/admin ghi sẵn trong mã và README, trả str(e) cho client.
S-11	Trung bình	auth.py	Đổi tên đăng nhập và tiểu sử không cần xác thực hay mật khẩu.
S-12	Thấp	Frontend	Phân quyền chỉ ở phía client (route /admin và vai trò đọc từ localStorage).
S-13	Thấp	schema.sql	reviews và reviewImages được khai báo hai lần khác nhau; IF NOT EXISTS làm bản đầu thặng. Bản đầu có reviewImages.id không tự sinh nhưng mã chèn không cung cấp id , nên thêm ảnh đánh giá gặp lỗi 1364 ở chế độ strict mặc định của MySQL. Bản đầu của reviews cũng thiếu UNIQUE(userId, productId) của bản sau.
S-14	Thấp	Nhiều route	request.json có thể là None ; URL ảnh không kiểm tra; int() trên đầu vào không bắt lỗi.

Liên quan trực tiếp tới Web3: nếu không sửa S-01, bước liên kết ví (BR-07) vô nghĩa vì kẻ tấn công có thể liên kết ví của mình với tài khoản người bán rồi nhận tiền; S-05 cho phép đổi trạng thái chiều của đơn Web3; S-02 và S-03 cho phép chiếm quyền Admin, tức quyền tạo đề xuất phán quyết.

16.3. Giai đoạn 0: nền tảng bắt buộc trước khi tích hợp

- Lớp xác thực (Q-10, đã chốt).** Đăng nhập trả token ký có hạn (ví dụ itsdangerous với SECRET_KEY bắt buộc đặt từ môi trường, hết hạn 8 giờ); frontend gửi qua header Authorization.Decorator require_auth(roles=[...]) đặt g.user ; mọi userId trong thân hoặc đường dẫn bị bỏ và lấy từ g.user . Đánh đổi: token ở bộ nhớ trình duyệt chịu rủi ro XSS, chấp nhận được ở mức đồ án.
- Sửa bốn lỗi nghiêm trọng (S-01 đến S-04):** xác thực mọi route, chỉ cho phép đăng ký user và seller , bảo vệ admin_bp bằng vai trò admin, bỏ cấp số dư cho người dùng (chỉ seed hoặc Admin).

3. **Kiểm quyền sở hữu (S-05, S-06, S-11):** đơn thuộc người mua, sản phẩm và shop thuộc người bán, cập nhật đơn kiểm trạng thái hợp lệ.
4. **Loại bỏ /api/shops/migrate** và thay bằng bộ chạy migration số thứ tự (S-07, mục 7.6); dọn schema: gộp khai báo trùng của `reviews` và `reviewImages`, giữ `UNIQUE(userId, productId)` và `reviewImages.id` tự tăng (S-13).
5. **Thanh toán an toàn (S-08, S-09):** giao dịch với `SELECT ... FOR UPDATE, UPDATE ... WHERE inStock >= :qty` và `balance >= :amount`, tính tiền bằng số nguyên VND hoặc `Decimal` (không dùng `float`), PIN chưa thiết lập trả 401 thay vì lỗi 500, giới hạn số lần thử đăng nhập và PIN.
6. **Cấu hình (S-10):** tắt debug theo môi trường, lỗi chung cho client và chi tiết vào log, đổi mật khẩu admin ban đầu khi cài đặt.
7. **Kiểm thử:** mỗi endpoint có ca không đăng nhập (401), sai vai trò (403) và truy cập đơn của người khác (403 hoặc 404); thêm vào CI hiện có (`pytest`, GitHub Actions).
8. **Chuyển sang VND (Q-09, DD-12):** đổi bốn cột tiền (`products.unitPrice`, `orderLines.unitPrice`, `orders.totalAmount`, `paymentMethods.balance`) sang `DECIMAL(15,0)`, dữ liệu cũ nhân với `FX_VND_PER_TOKEN`; thay ký hiệu `$` và `toFixed(2)` ở Navbar, Home, ProductView, Cart, Checkout, MyOrders, SellerDashboard bằng một hàm định dạng VND duy nhất (`Intl.NumberFormat` với `vi-VN` và `VND`); ô nhập giá của người bán chỉ nhận số nguyên không âm; thông báo của backend và số dư giả lập ban đầu đổi sang VND.

16.4. Ảnh hưởng tới kế hoạch 10 tuần

Bước 1 và 2 phải xong trước khi làm UC-00 (liên kết ví, tuần 3) và nên bắt đầu ngay tuần 1 bởi thành viên B, vì mọi API Web3 phụ thuộc vào đó. Bước 3 đến 7 chia cho thành viên A khi xong M1 (tuần 3-4), cùng phần relay và thư viện EIP-712 đã giao. Tuần 3-6 vốn đã là điểm nghẽn của B, nên nếu không đủ người thì cắt bớt tính năng phụ như BRD mục 9 đã nêu, không cắt bước 1 và 2.

Bước 8 (VND) đụng cả backend lẫn frontend nên giao cho B và C ở tuần 1-2, song song với bước 1 và 2. Nạp gom (4.9) hoàn thiện cùng contract ở tuần 2 (M1); phần backend và giao diện của lô nằm trong UC-01 và UC-02 từ tuần 3.

Mọi mã Solidity và cấu hình trong SRS vẫn là mã đồ án chưa qua kiểm toán độc lập; tài liệu này không thay thế kiểm toán.